

무엇을 남기고 무엇을 잊을 것인가: LLM과 에이전트 기억 압축의 율-왜곡 관점

What to Keep, What to Forget: A Rate – Distortion View of Memory Compaction in LLMs and Agents

Ashwin Gerard Colaco, Nada Lahjouji

문서 성격 원문 순서 · 수치 · 한정어를 보존한 한국어 구조 보존형 의역본이다. 본문 뒤의 원문 보존 부록은 원 논문의 equation, table, figure, caption 을 검증 가능하게 남긴다. 인용과 재현에는 반드시 고정 원문을 사용한다.

고정 버전 arXiv:2607.08032v1

배포 상태 internal-only — arXiv license; translated derivative and complete source plate are not cleared for public redistribution

Contents

초록 (Abstract)	2
1. 서론 (Introduction)	2
2. 통합된 Rate – Distortion 정식화	2
3. 일곱 축 Taxonomy	3
4. KV-cache Compaction	3
4.1 Eviction	3
4.2 Quantization	3
4.3 Low-rank, hidden-dimension, sharing 과 merging	3
5. Prompt 와 Context Compaction	3
6. Architecture-level Bounded Memory	3
7. Agent Long-Term Memory Compaction	4
8. Trainable Sparse Attention	4
9. Cross-layer Compaction	4
10. 계층 사이클 잇는 다섯 knob	4
11. System 과 Serving Substrate	4
12. Compaction 이론	5
13. 평가와 COMPACT-Bench	5
14. Reference Experiment	5
15. Open Problems and Research Agenda	5
한계 (Limitations)	6
16. 결론 (Conclusion)	6
번역 · 검증 고지	7
원문 구조 · 그림 보존 부록	8

초록 (Abstract)

LLM 과 그 위의 agent 는 KV cache, 긴 prompt, recurrent state, session history 를 유지하는데 점점 더 많은 compute 와 memory 를 쓴다. KV-cache compression, prompt compression, bounded architecture, agent-memory consolidation 은 서로 다른 연구 분야처럼 발전했지만, 이 논문은 모두하나의 **rate – distortion** 문제라고 주장한다. 제한된 budget 아래 context 에서 유래한 정보 중 무엇을 어떤 fidelity 로 남겨 downstream utility 를 보존할 것인가라는 문제다.

저자들은 하나의 compaction objective 와 layer-agnostic lower bound, 일곱축 taxonomy 를 제시한다. 분야 전반에서 importance signal 은 attention magnitude 또는 recency 에 치우쳐 있고, query 가 알려지기 전에 되돌릴 수없이 버린 정보가 나중에 필요해지는 공통 실패를 보인다고 정리한다. single-turn compression 은 많이 측정되지만 agent 가 실제로 겪는 반복 compaction 의 오차 누적은 거의 측정되지 않는다. 이를 위해 COMPACT-Bench, 작은 reference experiment, open-problem agenda 를 제안한다.

1. 서론 (Introduction)

긴 transformer 의 KV cache, 길어진 prompt, fixed recurrent state, multi-session agent store 는 서로 다른 위치에 있지만 모두 history (H) 를 더 작은 representation (Z) 로 바꾼다. compaction operator 를 (C), compacted representation 을 사용하는 operator 를 (U) 라 두면 다음과 같다.

$$Z = C(H; B), \quad \hat{Y} = U(Z, Q).$$

B 는 byte, token, state dimension, stored fact 수 같은 budget 이다. 핵심 평가는 compression ratio 만이 아니라 원 history 를 썼을 때의 task output Y 와 compacted output $\hat{Y}$ 사이 distortion 이다. 같은 budget 에서 query 에 맞추어 필요한 정보를 다시 가져올 수 있는지, 되돌릴 수 있는지, 여러 fidelity 를 유지하는지가 성능을 좌우한다.

2. 통합된 Rate – Distortion 정식화

논문은 expected task loss 와 storage/compute cost 를 결합한다.

$$\min_C \mathbb{E}_{Q,H} [d(Y, U(C(H), Q))] + \lambda R(C(H)), \quad R(C(H)) \leq B.$$

query-conditioned information requirement 를 $I^{\wedge\{\star\}}(Q) = I(Y; H|Q)$ 라고 둔다. query-agnostic compactor 가 (B) bit 이하 representation 을 만들면 data-processing inequality 와 Fano inequality 로 error lower bound 를 얻는다.

$$P_e \geq \frac{H(Y|Q) - B - 1}{\log |\mathcal{Y}|}, \quad B < I^{\star}(Q).$$

이 bound 는 KV vector, gist token, recurrent state, agent note 처럼 representation 종류와 무관하다. 필요한 정보량보다 budget 이 작으면 어떤 compactor 도 error 를 피할 수 없다. query 를 모른 채 압축하면 여러 가능한 (Q) 를 대비해야 하므로 더 큰 rate 가 필요하다.

논문은 세 속성을 강조한다. **P-rev** 는 버린 원 정보를 다시 가져오거나 compaction 을 되돌릴 수 있는가, **P-q** 는 query 를 본 뒤 선택을 조정할 수 있는가, **P-fid** 는 lossless leaf 와 lossy summary 같은 여러 fidelity 를 함께 유지하는가이다.

3. 일곱 축 Taxonomy

방법을 다음 축으로 분류한다: 압축 unit(bit, token, head, span, vector, fact), lifecycle 위치 (prefill, decode, serving, task 내부, task 사이), budget 종류, query-conditioning 시점, reversibility, fidelity level, learned/training-free 여부. 서로 다른 논문이 보고하는 “4 배 compression” 은 unit 과 metric 이 다르면 직접 비교할 수 없으므로 byte/token 과 downstream accuracy 를 함께 정규화해야 한다.

4. KV-cache Compaction

4.1 Eviction

token 또는 head 의 importance 를 attention score, recency, heavy hitter 로 추정해 KV entry 를 버린다. decode memory 와 bandwidth 를 줄이지만 query 가 진행되며 중요도가 바뀌면 이미 버린 token 을 복구하기 어렵다. prefix 와 recent window 를 보존하는 heuristic 은 많은 task 에 유용하지만 “중간의 특정사실” 을 나중에 묻는 경우 실패할 수 있다.

4.2 Quantization

모든 token 을 남기되 key/value bit-width 를 낮춘다. eviction 보다 P-rev 에 가깝지만 quantization error 와 outlier 처리가 필요하다. 같은 memory budget 에서 token 수와 bit fidelity 사이를 교환한다.

4.3 Low-rank, hidden-dimension, sharing 과 merging

KV matrix 를 factorize 하거나 head/layer 사이 representation 을 공유하고 유사 token 을 merge 한다. 구조적 redundancy 를 이용하지만 어떤 subspace 가 future query 에 필요한지 모르면 rare direction 을 손실할 수 있다. 논문은 output-error bound 가 있는 방법을 agent summary 의 stopping rule 로 옮길 가능성을 제시한다.

5. Prompt 와 Context Compaction

extractive 방법은 중요 span 을 선택해 원문을 보존하고, abstractive 방법은 더 짧은 text 로 다시 쓴다. learned prompt compressor 와 gist vector 는 token 수를 크게 줄이지만 사람이 읽고 복구하기 어려울 수 있다. query-aware selection 은 특정 질문에는 효율적이지만 동일 compact context 를 다른 질문에 재사용하기 어렵다.

lossy summary 를 반복해서 다시 summary 하면 작은 오류와 누락이 다음 단계 입력이 되어 self-reinforce 될 수 있다. 원 chunk 를 leaf tier 에 남기고 위에 hierarchical summary 를 만드는 RAPTOR·GraphRAG 유형은 multi-fidelity 와 reversibility 를 일부 제공한다.

6. Architecture-level Bounded Memory

segment recurrence, linear attention, state-space model 은 history 를 fixed-size state 에 압축해 ($O(L)$) 또는 subquadratic compute 를 얻는다. budget 이 sequence length 와 함께 늘

지 않으므로 장기 recall 에서 information bottleneck 이 생긴다. Memory Caching 처럼 과거 state checkpoint 를 추가하면 capacity 를 늘릴 수 있지만, 결국 cache 수와 access cost 라는 새 budget 이 생긴다.

bounded architecture 는 compaction operator 가 model training 안에 포함된다는 장점이 있다. 반면 state 가 어떤 정보를 잃었는지 inspect·rollback 하기 어렵고, query 가 나중에 도착하면 write 시점의 query-agnostic compression 을 되돌릴 수 없다.

7. Agent Long-Term Memory Compaction

agent 는 trajectory 를 trim·summarize 하고, session 사이 fact 를 text/vector/graph 에 저장한다. MemGPT 는 tier 간 paging, Mem0 는 extraction/update, Zep 은 temporal knowledge graph, hierarchical RAG 는 multi-level summary 를 사용한다. raw episode 를 남기는 design 은 P·rev 를 높이고, derived fact 만 남기는 design 은 compact 하지만 extraction error 에 취약하다.

agent memory 의 고유 문제는 동일 memory 가 여러 번 consolidate 된다는 점이다. 처음에는 정확한 note 도 이후 summary·merge·conflict resolution 을 거치며 변한다. single-shot recall benchmark 는 이 compounding curve 를 측정하지 않는다.

8. Trainable Sparse Attention

training-free KV heuristic 은 pre-trained weight 가 임의의 missing token 을 견딜 것이라 기대한다. trainable sparse attention 은 어떤 token/head 를 남길지 학습하여 compaction 과 model 을 공동 최적화한다. learned selector 가 training query distribution 에 과적합하면 새로운 query 에서 동일 failure 가 날 수 있고, hard selection 의 reversibility 문제는 남는다.

9. Cross-layer Compaction

실제 system 은 prompt 를 먼저 줄이고, prefill 뒤 KV 를 quantize/evict 하며, agent turn 사이 summary 를 만든다. 각 layer 를 따로 최적화하면 같은 information 을 중복 보존하거나 모두 버릴 수 있다. 논문은 C1 부터 Ck 까지 이어지는 operator composition 의 distortion 을 공통 budget 으로 측정해야 한다고 주장한다. 현재는 이를 지배하는 일반 composition law 가 확립되지 않았다.

10. 계층 사이를 잇는 다섯 knob

unit, timing, query adaptivity, reversibility, fidelity 라는 공통 knob 를 쓰면 한 분야의 mechanism 을 다른 분야로 옮길 수 있다. 예를 들어 query-aware KV retrieval 은 agent store 에서 “미리 삭제하지 말고 cheap index 로 후보를 좁힌 뒤 query 에 맞춰 원문을 fetch” 하는 design 으로 대응한다. KV output-error bound 는 summary 를 언제 멈출지 정하는 threshold 가 될 수 있다.

11. System 과 Serving Substrate

compaction 은 GPU HBM, host memory, SSD/object store, network 사이 data movement 를 바꾼다. reversible design 은 lower tier 의 lossless copy 와 index 를 요구하고, query-time

fetch 가 latency tail 을만든다. multi-fidelity tier 는 hot summary 와 cold raw episode 의 consistency/version 을 관리해야 한다.

batching 과 cache reuse 도 중요하다. session 별 state 를 자주 merge-rewrite 하면 immutable prefix sharing 이 깨질 수 있다. asynchronous compaction 은 critical path 를 피하지만, 완료 전 · 후 version 을 query 가 일관되게 보도록 publication protocol 이 필요하다.

12. Compaction 이론

lower bound 의 ($I^{\star}(Q)$) 는 실제 task 에서 직접 알기 어렵다. attention magnitude 와 recency 는 이 값을 추정하는 proxy 일 뿐이다. 논문은 marginal task utility 에 따라 budget 을 할당하고, query distribution 이변할 때 다시 추정하는 문제를 제시한다. multi-layer budget 은 각 unit 의 residual information need 가 균형을 이루도록 배분하는 것이 이상적이지만실용 estimator 와 guarantee 가 없다.

13. 평가와 COMPACT-Bench

COMPACT-Bench 는 서로 다른 layer 의 방법을 동일 accuracy – budget curve 에놓고, latency 와 byte 를 함께 기록하며, single-shot 과 repeated compaction 을 분리하도록 설계된다. query 가 compression 전에 알려지는조건과 나중에 알려지는 조건, reversible fetch 허용 여부, source distribution shift 를 교차한다.

평가에는 exact recall, multi-hop reasoning, temporal update, abstention, end-task utility 와 함께 source fidelity·rollback 을포함한다. 단일 compression ratio 가 아니라 Pareto frontier 와 error accumulation slope 를 보고해야 한다.

14. Reference Experiment

저자들은 commodity GPU 의 small open model 에서 KV eviction, quantization, prompt compression, summarization 을 하나의 budget axis 에 놓는 reference-scale 실험을 수행한다. 절대 수치는 방법론의 예시이며 대규모 production result 로 제시되지 않는다. repeated irreversible summarization 은 cycle 이 늘며 error 가 커지고, retrieval-backed reversible condition 은 더 평평한 curve 를 보이는데를 시험한다.

15. Open Problems and Research Agenda

우선순위에는 모든 layer 의 query-conditioned reversible multi-fidelity compaction, cross-layer budget allocation, repeated-compaction benchmark, lossless episodic tier 에서 lossy semantic tier 로의 promotion rule, cheap asynchronous self-curation, RL-stable memory policy, guarantee 가 있는 adaptive allocation, evolving memory 의 audit·rollback·revocation 이 포함된다.

논문은 sleep-time compute 를 critical path 밖의 asynchronous curation 사례로 언급한다. 중요한 조건은 단지 밤에 계산한다는 것이 아니라, raw source 를 복구 가능하게 남기고, 어떤 query distribution 을 위해 무엇을 lossy 하게 승격했는지 기록하며, repeated error 를 측정하는 것이다.

한계 (Limitations)

이 연구는 survey 와 formalism, benchmark proposal 이며 대규모 empirical validation 을 제공하지 않는다. lower bound 의 가장 단순한 형태는 query-agnostic compactor 를 가정하고 $(I^{\star}(Q))$ 를 주어진 값으로 취급한다. 실제 task 의 information need 를 추정하는 일 자체가 open problem 이다. cross-layer operator algebra 는 아직 증명된 law 가 아니라 empirical composition map 이다. reference experiment 는 single commodity GPU 의 작은 model 이라 절대 성능은 scale evidence 가 아니다. 최신 preprint 가 많아 독립 재현을 기다리는 수치도 있다.

16. 결론 (Conclusion)

KV token eviction, quantization, prompt pruning, bounded recurrent state, agent trajectory summary 는 모두 budget 아래 task utility 를 보존하는 rate – distortion decision 이다. 같은 budget 에서 되돌릴 수 있고 query 에 맞춰 선택하는 operator 가 유리하며, 반복 lossy compaction 의 누적 error 를 공통 benchmark 로 측정해야 한다. 뒤 원문 부록은 taxonomy, bound, survey table, COMPACT-Bench 와 reference experiment 전체를 보존한다.

번역·검증 고지

이 문서는 분석이나 평가를 추가하지 않는 것을 원칙으로 하며, 원문의 주장 강도와 수치를 유지하도록 작성되었다. 한국어 초안 작성과 구조 검수에는 Codex GPT-5.6 을 사용했고, 고정 source hash·원문 보존 부록·역대조 표를 통해 검증했다. 불일치가 있을 때에는 원문이 우선한다.

원문 구조·그림 보존 부록

아래 부록은 고정 원문의 equation, table, figure, caption 과 section 배치를 대조하기 위한 검증판이다. arXiv license; translated derivative and complete source plate are not cleared for public redistribution

What to Keep, What to Forget: A Rate–Distortion View of Memory Compaction in LLMs and Agents

Ashwin Gerard Colaco
acolaco@uci.edu

University of California, Irvine
Irvine, California, USA

Nada Lahjouji
nlahjouj@uci.edu

University of California, Irvine
Irvine, California, USA

Abstract

Large language models, and the agents built on them, spend an ever-growing share of their compute and memory on remembering: caching attention keys and values, carrying long prompts, maintaining recurrent state, and storing what happened in previous turns and sessions. Because none of this memory is free, four largely separate research communities have each learned to compact it. They evict or quantize the KV cache, prune or distill prompts, bound architectural state, and consolidate agent memory. We argue that these are instances of one problem: a rate–distortion decision about **what context-derived information to retain versus discard, at what fidelity, under a resource budget**, so as to preserve downstream task utility. We make this lens precise with a single compaction objective and a layer-agnostic lower bound, use it to build a seven-axis taxonomy that classifies methods from across the stack uniformly, and use it to transfer mechanisms between layers that have never been connected, from serving-stack KV management to agent long-term memory. Two patterns hold across the survey. At every layer the signal that decides what to keep is attention magnitude or recency, and it fails in the same way everywhere, by discarding, before the query is known and with no way to undo it, information the query later needs. And while compression is measured carefully on single-turn long context, the repeated compaction that agents actually perform is almost never measured, and no benchmark holds one budget axis across all the layers at once. We turn both observations into a benchmark proposal, a small reference experiment, and a set of compaction-aware design principles, and we map the open problems.

CCS Concepts

• **Computing methodologies** → **Natural language processing; Machine learning**; • **Information systems** → *Information retrieval*.

Keywords

memory compaction, KV cache compression, prompt compression, long-context language models, agent memory, rate–distortion, information bottleneck, efficient inference

1 Introduction

Run a transformer over a hundred thousand tokens and the cached keys and values of everything it has already read take up more of the accelerator than the model’s own weights [56, 72]. The same pressure appears in other forms. An agent working toward a goal builds up a record of tool calls and their results that soon outgrows any context window [86, 132]; a state-space or linear-attention

model squeezes the whole past into a state of fixed size; an assistant expected to know its user has to carry something from one session into the next [74, 118]. In each case the scarce resource is no longer computation but the memory of what has been read, and the practical question is what to do when there is more of it than fits.

Four bodies of work have grown up around that question with little contact between them. **KV-cache compression** acts during inference: it drops low-value tokens [60, 123, 142], stores the rest at a few bits [45, 72], factorizes them [12, 28], or merges them [67, 140]. **Prompt and context compression** acts earlier, before the model reads the input, deleting or rewriting tokens [49, 59] or swapping spans for a few learned “gist” vectors [19, 39, 80]. A third line changes the **architecture** so that memory is bounded by construction, through segment recurrence [10, 93], a fixed compressive memory [81], or linear-time state-space models [6, 40, 90]. A fourth works at the scale of a whole **agent**, trimming the working context during a task [53, 105, 148] and consolidating what is worth keeping across tasks [21, 88, 125]. Each line carries its own benchmarks and its own idea of success, whether that is compression ratio at fixed perplexity, tokens saved at fixed accuracy, or recall at fixed state size, and a method from one line seldom cites a method from another.

These four are the same problem in different clothes. A method at any level takes the history H of everything the model has read and produces a smaller representation Z that fits a budget B ; call this the *compaction operator* C . The model then runs on Z in place of the original, through a *usage operator* $\mathcal{U}$. Stated or not, the method is solving one optimization: keep Z small while giving up as little downstream accuracy as possible. That is a rate–distortion problem (Section 2), and once the field is read this way the differences between methods come down to three. What unit do they compress: a bit, a token, an attention head, a span of text, a vector, a stored fact? Where in the model’s life do they act: pretraining, prefill, decoding, serving, inside a task, or between tasks? And do they decide what to keep before the query is known, or after? Figure 1 maps the survey onto these distinctions.

Reading the field this way buys three things. The first is a shared yardstick. A KV evictor that keeps a quarter of the tokens and a quantizer that stores every token at four bits land at the same point on a bytes-per-token axis, so they can be compared on the accuracy each buys there, and an agent’s summary can be placed on that axis too. The second is that one failure mode turns out to lie behind many symptoms. A method that fixes what it keeps before the query arrives, and cannot take the choice back, will sooner or later drop something the query needed, whether it is evicting KV entries, summarizing a transcript, or folding the past into a recurrent state.

The two properties at work, reversibility and query-conditioning, recur throughout. The third is that techniques become portable: once an agent’s Ebbinghaus-style forgetting curve [146] and a KV evictor’s attention-score rule [142] are seen as two estimates of the same quantity, the better-studied one is a design for the other, the no-eviction retrieval of Quest [108] suggests an agent-memory design, and the output-error bound of Ada-KV [34] suggests a stopping rule for summarization (Section 10).

We are not the first to connect these areas. One recent survey gathers token-level, parametric, latent, and agent memory under a Forms/Functions/Dynamics scheme [47], and another casts agent memory as a write–manage–read POMDP [31]. What this survey adds is a single optimization objective with a provable bound rather than a descriptive grouping; coverage that reaches the serving stack and the quantization and low-rank mathematics alongside agent memory, which no prior survey spans; the explicit transfer of mechanisms between layers; and a benchmark we both define and run. Table 1 sets out the comparison.

We make six contributions.

- We formalize memory compaction as one rate–distortion problem and derive a layer-agnostic lower bound (Eq. 2) showing that, below a task’s information requirement, every layer must err at a rate set by the same expression, and that query-agnostic compaction pays a quantifiable penalty (Section 2).
- We give a seven-axis taxonomy (Section 3) that classifies methods from all four layers uniformly, and a master table.
- We survey each layer through the formalism: KV cache (Section 4), prompt/context (Section 5), architectural (Section 6), agent/semantic (Section 7), the trainable sparse-attention frontier (Section 8), and multimodal/multi-agent settings (Section 9).
- We make the inference↔agent-memory bridge explicit (Section 10), porting concrete mechanisms across layers and deriving five compaction-aware design principles.
- We propose **COMPACT-Bench** and run a small reference experiment on commodity hardware that places KV eviction, quantization, prompt compression, and summarization on one budget axis and measures error accumulation under repeated compaction (Sections 13–14).
- We consolidate the open problems into a prioritized agenda (Section 15).

Two themes return throughout. One is that reversibility usually matters more than any scoring trick: at the same budget, a method that can fetch back what it discarded beats one that cannot, and it wins precisely on the queries that turn on the discarded material. The other is a measurement gap. Compaction is tested on single-turn long-context tasks, but agents compact the same memory again and again, and almost nothing measures what that repetition costs; Section 13 proposes a benchmark that does.

We restrict attention to the compaction of memory that comes from a model’s context, drawing mostly on work from 2023 to 2026 in the machine learning, natural language processing, and systems literatures, preprints included. A method is in scope if it shrinks the memory or token footprint of the context while trying to keep

the model’s behavior intact. Weight-level compression (pruning, distillation, weight quantization) is out of scope, as is long-context training that imposes no memory bound, except where such work builds in a bounded memory. Much of the literature is recent and unrefereed, so we treat preprint numbers as provisional and flag results that have not been reproduced.

2 A Unified Formalism for Compaction

This section turns the informal claim into something we can compute with: an objective, a lower bound that holds across all four layers, a reading of the field’s “importance” heuristics as estimates of one quantity, and the two properties that most often decide whether a method fails.

Fix a point during generation, and let H stand for everything the model can draw on there: the prompt tokens and their activations, the KV cache, any recurrent state, and any memory of prior turns. A *compaction scheme* is a pair $(C_\theta, \mathcal{U})$. The compaction operator $C_\theta : H \mapsto Z$ produces a compact representation Z whose size obeys a budget, $\text{rate}(Z) \leq B$, where $\text{rate}(\cdot)$ measures memory in a currency appropriate to the layer (GPU bytes for a KV cache, tokens for a prompt, state dimensions for a recurrent model, store size for an agent). The usage operator $\mathcal{U}$ consumes Z , almost always by continuing the model’s computation, to produce an output $\hat{Y}$. For a task instance with query Q and target Y , the scheme’s quality is its expected loss, and the design problem is

$$\min_{\theta} \underbrace{\mathbb{E}_{(H,Q,Y)}[\ell(\mathcal{U}(C_\theta(H), Q), Y)]}_{\mathcal{D}(\theta) = \text{distortion}} \quad \text{s.t.} \quad \text{rate}(Z) \leq B. \quad (1)$$

Equation (1) is a rate–distortion problem: trade memory rate against task distortion. Its information-bottleneck form makes the trade explicit, namely maximize $I(Z; Y | Q)$ subject to $I(Z; H) \leq B$ [109, 115]: keep the bits of the history that predict the answer, spend nothing on the rest.

The objective is not vacuous, and its consequences are the same at every layer. Let $I^*(Q) = I(Y; H | Q)$ be the *task-conditioned information content*: the number of bits of H genuinely required to answer Q . Because $\hat{Y} = \mathcal{U}(Z, Q)$ is a function of (Z, Q) and, for a query-agnostic C , the chain $Y - H - Z$ is Markov given Q , the data-processing inequality gives $I(Y; \hat{Y} | Q) \leq \min(I^*(Q), B)$. Fano’s inequality then bounds the error probability $P_e = \Pr[\hat{Y} \neq Y]$ over an answer space $\mathcal{Y}$:

$$P_e \geq \frac{H(Y | Q) - B - 1}{\log |\mathcal{Y}|}, \quad \text{whenever } B < I^*(Q). \quad (2)$$

The bound holds for any compact memory Z , whether a KV cache, a gist vector, a recurrent state, or an agent’s note store, which is what lets it unify the field. Three consequences follow directly. (i) Below the task’s information requirement $I^*(Q)$, every layer must make errors, at a rate governed by the same expression; there is no architecture-specific escape. (ii) The compression ratio achievable at fixed quality is task-dependent, $\propto 1/I^*(Q)$: tasks whose answers carry many bits (exact or multi-hop retrieval) compress poorly, while tasks whose answers are low-entropy (classification, gisting of redundant prose) compress well. This is exactly why the same KV evictor that is near-lossless on summarization collapses

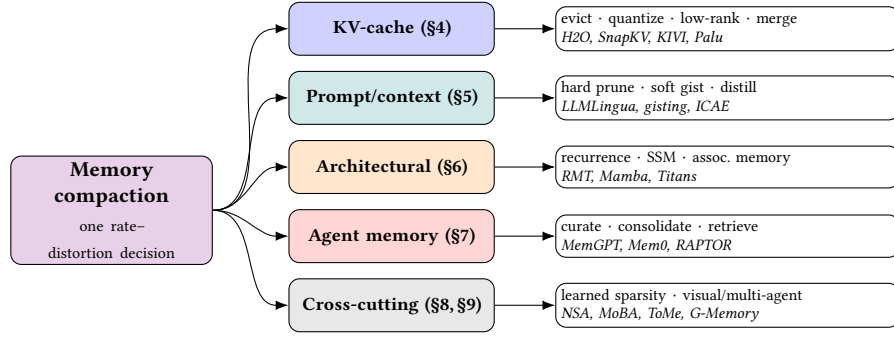


Figure 1: Map of the survey. The four areas of memory compaction act on different substrates and at different times, yet each makes the same rate–distortion choice (Section 2): the KV cache (Section 4), the prompt and context (Section 5), architectural state (Section 6), and agent memory (Section 7), with trainable sparse attention (Section 8) and multimodal or multi-agent settings (Section 9) cutting across them. The taxonomy of Section 3 classifies the methods on shared axes, and Section 10 carries mechanisms between them.

Survey	KV	Prompt	Arch.	Agent	Sys.	RD/IB objective	X-layer transfer	Unified benchmark
Memory in the Age of AI Agents [47]	●	●	●	●	○	○	●	○
KV Cache Management [58]	●	○	○	○	●	○	○	○
KV Cache Compression: A Review [69]	●	○	○	○	●	○	○	○
Prompt Compression [62]	○	●	○	○	○	●	○	○
Memory for Autonomous Agents [31]	○	●	●	●	○	●	●	●
Security of Agent Memory [66]	○	○	○	●	○	○	○	○
Efficient LLMs [111]	●	○	○	○	●	○	○	○
This survey	●	●	●	●	●	●	●	●

Table 1: Coverage of this survey relative to the closest prior surveys. ● dedicated treatment; ● discussed within another topic; ○ absent. Columns are the four compaction layers (KV cache, prompt/context, architectural/recurrent, agent/semantic), the serving/systems substrate, a unifying rate–distortion / information-bottleneck objective, explicit cross-layer mechanism transfer, and a unified compaction benchmark. Recent surveys span the first four columns (notably 47); our novelty is the last three.

on retrieval. (iii) A query-agnostic operator must spread its budget across the whole query distribution, so its effective per-query budget is roughly $B - H(Q)$, whereas a query-conditioned operator can spend all of B on the answer-relevant bits; the gap $H(Q)$ is the precise price of not knowing the query. The $\Theta(nd)$ lower bound on exact-attention memory [44] and the impossibility of exact in-context retrieval with fixed state [117] are special cases of this picture.

Importance heuristics are best understood as distortion surrogates. No method can evaluate $\mathcal{D}$ directly, because Y is unknown at compaction time. Each instead picks a *surrogate*, a cheap proxy for “how much will dropping this hurt the answer.” Seen through Eq. (1), the field’s apparently unrelated heuristics are all surrogate distortion estimators: accumulated attention mass (H2O, SnapKV; 60, 142), token self-information or perplexity (LLMLingua; 49), an explicit mutual-information estimate (QUITO-X; 115), a bound on the induced output error (Ada-KV; 34), and an LLM’s own judgment of salience (Mem0, reflection; 21, 88). Naming them as estimators of one quantity is precisely what makes them comparable, and transferable, across layers.

Three first-class properties separate methods that fail from those that do not. Two schemes can sit at the same budget B and the same average distortion yet behave completely differently on the queries that matter. The difference is captured by three properties that Eq. (1) leaves implicit and that we promote to the center of the survey. **(P-rev) Reversibility**: can content dropped by C be re-derived if a later query needs it? Retrieval-backed and archival schemes can; eviction and summarization cannot. **(P-q) Query-conditioning**: does C see the query (or its distribution) when it decides what to keep? Offline gisting does not; LongLLMLingua and Quest do. **(P-fid) Fidelity profile**: is the representation lossless, near-lossless, uniformly lossy, or multi-fidelity (a small high-fidelity tier plus a large lossy tier)?

The formalism yields four falsifiable predictions. The bound implies that violating (P-rev)+(P-q) produces the same failure at every layer, with a layer-specific, testable signature. **(1) KV**: for attention-score eviction, needle recall collapses once the per-token budget falls below the needle’s share of attention mass, and the collapse point tracks the entropy of the answer’s location, worsening in mid-context. **(2) Prompt**: query-aware compression shifts the

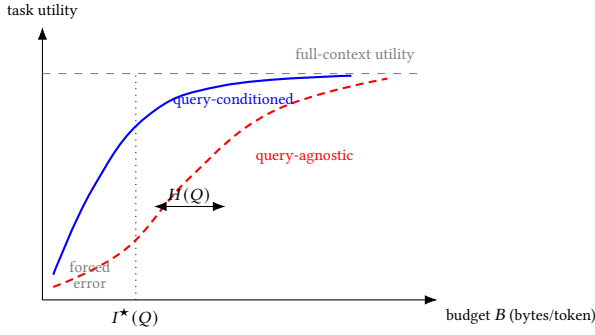


Figure 2: The rate–distortion view of Section 2. Below the task-conditioned information content $I^*(Q)$, no scheme can avoid error (Eq. 2). A query-agnostic operator (red) reaches a given utility only at a larger budget than a query-conditioned one (blue). The horizontal gap is the query entropy $H(Q)$, which the bound charges for not knowing the query in advance.

accuracy-versus-ratio curve to the right of query-agnostic compression by exactly the mutual-information gap. (3) **Architectural**: a state of s bits cannot answer a query requiring more than s bits, so state-space and linear-attention models exhibit a hard accuracy cliff on multi-key recall at a key count proportional to state size, while attention degrades gracefully. (4) **Agent**: under repeated *irreversible* summarization, end-task error grows super-linearly in the number of compaction events, whereas a reversible, retrieval-backed memory stays flat. We test (1) and (4) directly in Section 14.

3 A Seven-Axis Taxonomy

The formalism exposes the dimensions along which methods genuinely differ. We pull these out as seven axes, chosen to be as close to orthogonal as the design space allows. A method is a point in their product. As Section 2 predicts, a method’s failure mode follows from the combination of axis values, not from any single value read in isolation.

- (1) **Granularity of the compressed unit.** From finest to coarsest: bit-width (quantization), hidden dimension or rank (low-rank factorization), token / KV entry, page or block, layer or head, natural-language span, dense soft token, recurrent state, semantic item (a fact, note, or graph node), visual token, and inter-agent message.
- (2) **Lifecycle stage, where the operator acts.** Architecture and pretraining; prompt or prefill time, before the model reads the context; prefill-to-decode KV formation; decode-time dynamic selection; the serving runtime, across requests; within-task working-context curation; between-task consolidation; and offline corpus indexing.
- (3) **Lossiness and fidelity.** Lossless reuse, near-lossless approximation, uniformly lossy, or multi-fidelity (a small exact tier plus a large lossy tier); and, cross-cutting, reversible versus irreversible (P-rev).
- (4) **Query/task adaptivity.** Query-agnostic and offline-cacheable, query-conditioned and online, or task-aware via a learned

reward (P-q); together with the importance *signal* used (attention score, perplexity, mutual information, output-error bound, or LLM-judged salience), which the formalism identifies as competing distortion surrogates.

- (5) **Learnability.** Training-free heuristic, post-training adapter, trained-from-scratch architecture, RL-learned policy, or LLM-as-controller with no weight change.
- (6) **Mechanism.** Drop/evict, select/retrieve (keep all), merge/cluster, quantize/encode, factorize/low-rank, abstractive summarize/rewrite, encode-to-latent, recurrent write–forget, structure/graph build, or internalize-to-weights.
- (7) **Storage substrate.** On-GPU KV, a host/SSD/remote tier, model parameters, an external text or vector store, a knowledge graph, in-context dense vectors, or a hybrid parametric/non-parametric store.

Table 2 gives the characteristic axis values for each layer we survey. The differences sit in three columns: granularity, lifecycle, and adaptivity. These are the three dimensions the lens flags as decisive, and the remaining axes vary far less across layers. Appendix A carries the full per-method classification of roughly seventy methods.

4 KV-Cache-Level Compaction

The KV cache attracts more compaction work than any other layer. At long context and large batch it is the cache, not the weights, that dominates GPU memory and decode bandwidth, since every layer rereads every cached key and value at every step. The history H is the cache itself: per layer, per head, a key and value vector for each past token. The operator C shrinks $\text{rate}(Z)$ measured in GPU bytes, and $\mathcal{U}$ is ordinary attention over whatever survives. The dozens of methods below share one device, the *surrogate* they use for distortion. Nearly all of them estimate “how much will dropping or coarsening this entry hurt the answer” from the attention score, which Eq. (1) casts as a cheap distortion proxy, biased in the ways the bound predicts. We group methods by which axis of the cache they compress: the token (sequence) axis, the query-time read pattern, the bit-width, the hidden dimension, and the layer axis.

4.1 Token Eviction: Dropping Entries Irreversibly

Eviction permanently deletes a budgeted subset of tokens’ KV, exploiting the empirical sparsity of attention, in which a small set of tokens carries most of the mass. Every method here is irreversible (violates (P-rev)) and, with one exception, decides what to keep before the decode query is known (violates (P-q)); they are therefore the canonical instantiation of prediction (1).

The cheapest evictors ignore content entirely and rely on positional or structural policies. StreamingLLM [123] keeps the first few “attention-sink” tokens plus a sliding window of recent tokens and discards the middle, giving constant memory and stable infinite streaming for free; its cost is total amnesia about evicted middle context, so it fails any task whose evidence lies outside the window. The sink it exploits is itself now well understood as a structural property of softmax attention [41, 94]. FastGen [38] adds content

Layer	Granularity	Lifecycle stage	Typical fidelity	Adaptivity	Mechanism / substrate
KV-cache (§4)	bit, rank, token, head	prefill-to-decode, decode-time	near-lossless, usually irreversible	mostly query-agnostic	evict / quantize / low-rank / merge; on-GPU KV
Prompt/context (§5)	NL span, soft token	pre-model (prefill)	lossy (hard) or opaque (soft)	agnostic or query-aware	prune / summarize / encode; in-context
Architectural (§6)	recurrent state	pretraining	uniformly lossy, fixed size	learned	recurrent write-forget; parameters
Agent/semantic (§7)	fact, note, summary	within- and between-task	lossy, partly reversible	LLM- or RL-driven	summarize / structure; external store
Cross-cutting (§8, §9)	block, visual token, message	pretraining; pre-model; cross-agent	learned or highly lossy	learned / similarity-based	learned sparsity / merge; mixed substrate

Table 2: The layers we survey and their characteristic values on the seven axes (Section 3). All layers share the axes; what separates them is granularity, lifecycle stage, and whether the retain decision is query-conditioned, the three dimensions the unifying lens singles out as decisive.

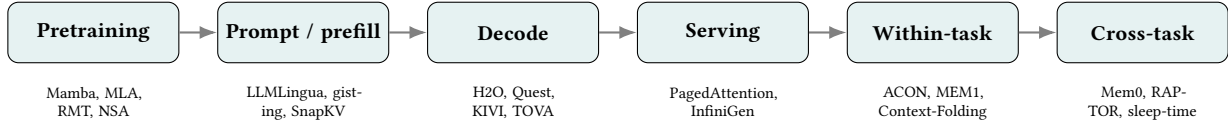


Figure 3: Where compaction acts across the model and agent lifecycle, the lifecycle axis of Section 3. The same retain-versus-discard decision recurs at each stage, on a different substrate and timescale; representative methods appear beneath each.

awareness without scoring: a one-shot profiling pass classifies each head as local, special-token, punctuation, or broad, and applies a per-head policy matched to that structure, recovering most attention at roughly half the memory but yielding only modest gains over fixed-budget scorers.

Most evictors instead score each token by its accumulated or recent attention and keep the top- B . H2O [142] casts eviction as greedy submodular maximization of accumulated attention, retaining a balance of recent and “heavy-hitter” tokens; Scissorhands [71] leans on a *persistence of importance* hypothesis, that tokens pivotal once stay pivotal, to fix a budget of historically high-attention tokens. Both read importance only from past steps. They therefore over-retain early, long-lived tokens and miss saliency that shifts with the (unseen) query. TOVA [85] reframes the decoder as a bounded multi-state RNN and drops the single lowest-current-attention token per step, removing recency assumptions but making a transient dip in attention permanently fatal. Keyformer [1] corrects the distribution shift that dropping induces with a Gumbel-softmax-regularized score, and SnapKV [60] introduces the influential *observation window*: it pools attention from the last prompt tokens to select, per head, the most-attended prompt tokens before generation. SnapKV works precisely when the query is already visible at prompt end, a partial, implicit form of query-conditioning, and degrades when it is not, exactly the gap of size $H(Q)$ that the bound assigns to not knowing the query.

A related question is not what to keep but where to spend a fixed total budget. PyramidInfer [126] and PyramidKV [11] allocate

layer-wise on a “pyramidal information funneling” observation, since attention is broad in lower layers and focused in higher ones. They give large budgets to shallow layers and small ones to deep layers, then apply SnapKV-style selection within each. Ada-KV [34] reallocates *across heads*: a layer-global top- k assigns each head a budget proportional to its share of the global selection, and it is the rare method that ties its surrogate to theory, deriving the allocation from a bound on post-eviction attention-output error rather than raw attention mass. These are allocation layers stacked on existing scorers; they improve budget utilization but inherit the underlying query-agnostic bias, and the layer profile remains a heuristic.

4.2 Dynamic Sparse Retrieval: Keeping Everything, Reading Selectively

A distinct strategy keeps the full cache in storage and instead makes the *read* sparse and query-conditioned at each step, restoring (P-rev) and (P-q) at the price of capacity savings. Quest [108] stores all KV in pages and, per decode step, estimates each page’s criticality from min/max key bounds against the current query, loading only the top- K critical pages; because no token is ever discarded and selection adapts to the query, it is near-lossless where eviction collapses. Landmark Attention [79] trains representative “landmark” tokens that gate random access to blocks of context, achieving the same effect through a learned index. These methods cut bandwidth, not capacity: they convert “what to forget” into “what to fetch,” which is why the bound’s $I^*(Q)$ penalty does not bind on them as it does on evictors.

4.3 Quantization: Compressing the Bit-Width

Orthogonal to which entries survive is how many bits each costs. Quantization is near-lossless in the bound’s terms, since it preserves (P-rev) as graceful fidelity loss rather than deletion, and is the most mature sub-field. KIVI [72] is the reference recipe: tuning-free asymmetric 2-bit quantization that quantizes keys *per channel* and values *per token* to match their distinct outlier structures, keeping a small full-precision residual window. KVQuant [45] pushes to 2–3 bits with per-channel, pre-RoPE, sensitivity-weighted non-uniform quantization plus dense-and-sparse outlier isolation, reaching million-token contexts on one GPU. Coupled Quantization [138] approaches one bit per channel by jointly encoding groups of statistically inter-dependent channels with a shared code. Atom [144] takes the systems view, quantizing weights, activations, and KV to 4 bits end-to-end with co-designed INT4 kernels so the memory saving becomes a throughput saving. The recurring obstacle, flagged across this family, is RoPE: rotary embeddings smear per-channel statistics, forcing the pre-RoPE quantization and decoupled handling that complicate kernels.

4.4 Low-Rank and Hidden-Dimension Compression

Compression can also target the per-token feature dimension. Palu [12] post-training low-rank-decomposes the K/V projections so only a smaller latent is cached and full keys/values are reconstructed on the fly, with a rank-search procedure and quantization compatibility, roughly halving the cache on existing checkpoints. GEAR [52] is explicitly compositional: ultra-low-bit quantization of the bulk, a low-rank matrix approximating the residual quantization error, and a sparse matrix for outliers, recovering much of the accuracy lost at 2 bits. Architecturally, Multi-head Latent Attention (MLA) in DeepSeek-V2 [28] *trains in* low-rank joint KV compression, caching one small latent up-projected at inference; it matches or beats MHA quality with a far smaller cache and has become the de facto standard for new frontier models, at the cost of pretraining from scratch and decoupled-RoPE handling. Bridging post-hoc and architectural, TransMLA [75] seeks cheap conversion of existing GQA checkpoints into latent attention, attacking the retrofitability gap.

4.5 Cross-Layer Sharing and Token Merging

Redundancy also runs along the layer dimension and within surviving tokens. xKV [13] performs post-training cross-layer SVD, consolidating a group of layers’ caches into a shared low-rank subspace by exploiting alignment of their dominant singular vectors, reaching up to 8× reduction. Architecturally, Cross-Layer Attention (CLA) [9] has only some layers compute K/V and the rest reuse earlier layers’ activations (orthogonal to MQA/GQA), and YOCO (You Only Cache Once) [106] caches KV exactly once via a decoder-decoder design in which a stacked cross-decoder reuses one global cache, saving roughly an L -fold factor. MiniCache [67] is the training-free counterpart, merging highly similar KV states between adjacent middle-to-deep layers by interpolating direction while preserving magnitude. A complementary idea merges rather than drops tokens: CaM [140] adaptively fuses to-be-evicted KV into surviving entries weighted by the discarded tokens’ attention, recovering much of eviction’s output perturbation, and KVMerger

Method	Mechanism	Gran.	Q-c.	Rev.	Reduct.
StreamingLLM [123]	positional evict	token	○	○	const. mem
H2O [142]	attn-score evict	token	○	○	~5×
SnapKV [60]	obs-window evict	head	●	○	~8×
PyramidKV [11]	layer-wise evict	layer	●	○	~8×
Ada-KV [34]	head-wise budget	head	●	○	varies
Quest [108]	sparse retrieval	page	●	●	read-only
KIVI [72]	2-bit quant	ch./tok	○	●	~2.6×
KVQuant [45]	non-unif. quant	channel	○	●	~5×
Palu [12]	low-rank proj	hidden	○	●	~2×
MLA [28]	latent attn	arch.	○	●	≫MHA
CaM [140]	token merge	token	○	●	add-on

Table 3: Representative KV-cache compaction methods by mechanism axis. Query-conditioning (Q-c.) and reversibility (Rev.): ● yes, ● partial, ○ no. Reductions are reported KV savings from the cited works and are not directly comparable across setups. Eviction methods are query-agnostic and irreversible, instantiating prediction (1); retrieval and quantization preserve reversibility.

clusters similar consecutive tokens into a pivotal token via Gaussian-kernel weights. Merging partially restores (P-rev) by averaging rather than deleting, but blurs distinct long-range information at small budgets.

4.6 Cross-Cutting Tensions

Three problems run through Table 3. The attention-score surrogate is systematically biased: accumulated scores over-retain early and high-frequency tokens and miss collectively-important marginal ones, which is why output-error-aware scoring (Ada-KV) and merging (CaM) have started to appear as corrections. RoPE interferes everywhere, confounding per-channel quantization and low-rank latents alike; that only some “retrieval heads” carry long-range dependence has motivated head-specific treatment (RazorAttention [107], ShadowKV [104]). The third problem matters most for our thesis. Methods that look near-lossless on summarization and Needle-in-a-Haystack fall off sharply on retrieval, multi-hop, long-generation, and reasoning/long-CoT workloads, which are exactly the high- $I^*(Q)$ tasks the bound says will resist compression; this gap has prompted decode-aware variants such as SCOPE [119]. Composition is the open frontier. Quantization, low-rank, and eviction attack orthogonal axes, yet whether their errors compound or cancel, and which combinations help, remains largely uncharted, and it is where a rate-distortion account should be doing the design work.

5 Context and Prompt Compaction

Prompt compaction works on the *input itself*, shrinking H before the target model attends to it. It chooses Z in the currency of input tokens or of soft embeddings that occupy token slots. KV eviction prunes inside a fixed forward pass; prompt compaction instead commits to a representation the target LLM must re-encode, so its design problem is Eq. (1) taken literally: maximize $I(Z; Y | Q)$ at a

token budget B . A dedicated survey already catalogs this literature by method family [62]. Our reading runs it through the IB objective of Section 2 and places it against the other layers, which surfaces the axis the field’s own taxonomy underplays: whether Z stays human-readable text or becomes opaque dense vectors. We split the section along that line.

5.1 Hard compaction: extractive and abstractive text

Hard methods keep Z in natural language, deleting low-value tokens or rewriting the context into fewer words. The output is text, so the target model stays untouched, the artifact ports across models, and a human can read it. That last property, inspectability, is reversibility’s weaker cousin. The shared move is to approximate the distortion in Eq. (1) with a *token-importance surrogate* from a small auxiliary model, then keep the high-importance tokens.

The first generation estimates importance from a language model’s own uncertainty, using self-information and perplexity surrogates. Selective Context scores each lexical unit by its self-information $-\log p(x_i | x_{<i})$ under a base LM and drops the least surprising tokens, phrases, or sentences [59]; LLMingua makes this iterative and budget-controlled, using a small LM’s token perplexity in a coarse-to-fine pass that protects entities and numbers while pruning predictable filler [49]. Self-information and perplexity are exactly the distortion surrogates named in Section 2: a token the model can already predict carries few bits of $I(Z; H)$ and is cheap to drop. Their weakness is that they score importance against the corpus prior rather than the query, a query-agnostic operator that, by consequence (iii) of Eq. (2), must spread B across the whole query distribution. LongLLMingua closes this gap by conditioning the surrogate on Q : it ranks content by question-aware contrastive perplexity, reallocates per-document budget, and reorders documents to fight lost-in-the-middle [50]. This is the cleanest empirical instance of prediction (2): moving from corpus-prior to query-conditioned scoring shifts the accuracy-versus-ratio curve to the right by the mutual-information gap $H(Q)$, which is why LongLLMingua can match or exceed the uncompressed prompt at ratios where LLMingua already degrades.

Perplexity scoring is slow and tokenizer-mismatched to the target, so the second generation replaces it with a learned classifier, distilled or RL-trained. LLMingua-2 reframes compaction as bidirectional preserve/discard token classification, training a small encoder on a GPT-4 distilled extractive dataset; it is faster and more faithful than perplexity pruning and is now the de facto task-agnostic baseline [87], but, supervised by a fixed teacher, it inherits a query-agnostic notion of importance. TACO-RL keeps the classifier but trains it with reinforcement learning against a task-specific reward (output divergence under a ratio constraint), turning a static surrogate into one optimized for the actual downstream loss in Eq. (1) [98]. The trajectory from Selective Context through LLMingua-2 to TACO-RL is the field walking its surrogate from a corpus-prior proxy toward the true task distortion, paying for each step with a more specialized, less transferable compressor.

QUITO-X makes the objective literal with an explicit information-bottleneck criterion. Rather than a perplexity proxy, it uses a cross-attention module to score each context token by its mutual information with the answer conditioned on the question, pruning to maximize the retained query-relevant $I(Z; Y | Q)$, which instantiates the IB form of Eq. (1) directly [109, 115]. It is the clearest evidence that the heterogeneous “importance” heuristics of this layer are all estimators of one quantity; the others differ only in how crudely they approximate the same MI.

A parallel line compacts retrieved evidence, where budget pressure is acute and the query is always present. RECOMP pairs an extractive selector with an abstractive summarizer trained for query relevance, emitting an empty summary when retrieval is unhelpful [124]; FILCO trains a sentence filter supervised by lexical and conditional cross-mutual-information signals [116]; Provence casts pruning as sequence labeling over the passage with a reused reranking head, making it near-zero-cost and self-calibrating in how many sentences to keep [22]; and CompAct condenses documents iteratively and actively for multi-hop QA, reaching extreme ratios at the cost of per-query generation and summarizer hallucination [134]. These span the granularity spectrum, from sentence-level filtering (FILCO, Provence) through single-pass abstractive summarization (RECOMP) to iterative active abstraction (CompAct), but are all query-conditioned by construction, which is why they sit far right on the accuracy-ratio curve. Their limit is faithfulness: abstractive variants give no guarantee that dropped facts were truly irrelevant, an irreversible loss no later query can recover (violating P-rev).

5.2 Soft compaction: learned latent representations

Soft methods let Z leave the token vocabulary: they train the context into a handful of dense vectors (gist tokens, memory slots, or compressed KV states) that occupy token slots but are continuous. Freed from the discreteness of text, they reach far higher ratios, but Z becomes opaque (no human can audit it) and typically query-agnostic and model-coupled.

The founding idea routes a prompt’s information through a few appended slots, an autoencoder from prompt to gist. Gisting trains the LLM during instruction tuning with a restricted attention mask that forces context to flow through “gist” tokens, yielding reusable, cacheable activations [80]; AutoCompressor recursively summarizes segments into accumulating summary vectors for very long inputs [20]; ICAE uses a LoRA-adapted copy of the LLM as an encoder that compresses context into memory slots decoded by the frozen model [39]; and 500xCompressor pushes to as few as one token by encoding into compressed KV (not output) representations, finding KV preserves more than embeddings at extreme ratios [63]. These are autoencoders for context: their reconstruction objective biases Z toward retaining *all* of H rather than the answer-relevant bits, which is why, by the task-dependence of Eq. (2), they are near-lossless on redundant prose yet shed exact-recall capacity sharply as the budget falls.

Some methods compress the per-layer cache rather than the input, distilling KV into latents. Activation Beacon interleaves beacon tokens that progressively compress each layer’s key/value activations, extending context to hundreds of thousands of tokens at

fixed quality [137]; KV-Distill distills a long KV cache into a short, question-independent one by treating the two caches as a student-teacher pair under a KL objective [14]. Being query-independent buys offline caching but forfeits the per-query budget advantage of consequence (iii); these methods pay the $H(Q)$ price by construction.

At the extreme, xRAG reinterprets a precomputed document retrieval embedding as a single modality token projected into the LLM’s space, with retriever and model frozen [19]: one vector, capacity hard-capped at the embedding. Context Distillation goes the opposite way, internalizing context into the weights for zero inference tokens, at the price of per-prompt retraining and no swappable artifact [102]. Cartridges generalizes this via offline per-corpus self-study, amortizing a long fixed corpus into a small trained KV “cartridge” reused across queries [33], a query-agnostic compaction whose cost is hidden from the serving path, akin to sleep-time compute [65].

Hard against soft trades interpretability and portability for compression ratio. Soft tokens buy the higher ratio with opacity and query-agnosticism, and they run a real risk: a single opaque vector can silently drop the bits a future query needs. The most rigorous study of gist compression confirms this: it is near-lossless on RAG and long-document QA but exhibits systematic failures (“lost by the boundary,” “lost if surprise,” and “lost along the way”) on synthetic recall, and its fine-grained mitigations narrow but do not close the gap to full attention [29]. That work also shows a plain average-pooling baseline often rivals trained gisting on long inputs: when H is information-dense, the learned compressor extracts little beyond what naive pooling captures, indicating the paradigm is not yet a drop-in for attention. Read through Eq. (2), these failures are the expected signature of a high- $I^*(Q)$ task met by a query-agnostic operator, the same wall the other three layers hit, here wearing soft-token clothing.

6 Architectural and Bounded-State Compaction

Sections 4 and 5 compact a representation the model was never trained to produce. They evict, quantize, or rewrite a cache the architecture builds at full size. The methods here instead bake compaction into the model and train it end to end, so the bounded state is the compact representation Z and the operators C_θ , $\mathcal{U}$ are learned jointly with the weights. They sit at the learned, trained-from-scratch end of the learnability axis, and they push the rate-distortion tension to its limit. When the state is a fixed s -bit object updated online, the bound of Eq. (2) bites directly: $B = s$ is fixed once, before any query is seen. This is prediction (3) of Section 2. A state of s bits cannot answer a query whose answer requires more than s bits, and what separates these methods is how each tries to spend s well.

The earliest architectural move keeps no compressed state at all; it caches. Transformer-XL caches the hidden activations of the previous fixed-length segment and reuses them as gradient-stopped keys and values for the current one, giving segment-level recurrence past the training window [23]. The cache is lossless but offers no compression: rate grows linearly with the number of cached segments, so the cache must be capped and older context dropped, a hard-edged, query-agnostic eviction (violating (P-rev)

and (P-q)). The Compressive Transformer adds a second, multi-fidelity tier (P-fid). Activations evicted from short-term memory are compressed at a fixed rate, via a strided convolution trained with an attention-reconstruction loss, into a smaller long-term store [93]. The Recurrent Memory Transformer (RMT) replaces caching with a true bottleneck. A small set of learned *memory* tokens carries information from one segment to the next, their outputs feeding back as the next segment’s memory inputs, with no change to the core block [10]. The number of memory tokens fixes s exactly, which makes RMT the cleanest instance of a learned, fixed-rate C_θ and the cleanest test of where the bound bites.

Another strand keeps the state lossy but makes the loss reversible (P-rev), retrieving from an external store instead of compressing into a bottleneck. Memorizing Transformers augment one attention layer with an approximate k -nearest-neighbor lookup into a large, non-differentiable memory of past key-value pairs, attending to retrieved history without backpropagating through it [121]. RETRO instead retrieves text chunks from a trillion-token database and fuses them through chunked cross-attention, decoupling knowledge from parameters [8]. Landmark Attention inserts a learned summary (“landmark”) token per block and trains attention to first select relevant blocks by their landmarks and then attend within them, giving random access through the attention mechanism itself [79]. The shared caveat, in the formalism’s terms, is that these methods sparsify retrieval but not storage. The full KV still lives somewhere, so the true rate(Z) is unchanged and the “compression” is of compute, not memory.

A third family makes the long-term store genuinely bounded. Infini-attention writes keys and values evicted from a local window into a fixed-size associative memory matrix by a linear-attention (delta-rule) update and reads them back with linear attention, fusing local softmax attention and a constant-size long-term memory in one block [81]. Memory is $O(1)$, but the read is the lossy linear-attention reconstruction that softmax avoids, and the write lacks a strong decay/forget gate. The method proved hard to reproduce, which the formalism reads as a missing content-aware write policy: it spends s without a surrogate for what to overwrite. Activation Beacon makes the same fixed-size move as a trainable plug-in on a frozen base, condensing each chunk’s keys and values into a few “beacon” activations via compression-based autoregression [137].

The dominant 2023–2025 strand treats the recurrent state itself as the compressor. Mamba replaces attention with an input-dependent (selective) state-space recurrence whose fixed-size hidden state summarizes all history, with a hardware-aware parallel scan for linear-time training and constant-state inference [40]. RWKV reaches the same constant-state regime as a linear-attention “RNN-Transformer” with a time-decayed key-value state [90]. Both make the rate-distortion trade explicit: the constant state is a lossy summary, so exact long-range copy and in-context recall lag softmax attention, which is prediction (3) made concrete. A delta-rule lineage refines how the state is written. DeltaNet replaces additive accumulation with a delta-rule (error-correcting) update that overwrites the value bound to a key, and parallelizes it across the sequence [130]; Gated DeltaNet adds a data-dependent forget gate atop the delta rule, combining principled erasure with decay [128]; GLA supplies hardware-efficient gated linear attention [129]; and xLSTM revives LSTM gating with exponential gates and a matrix

memory [5]. All four learn a write/forget policy, a surrogate for which bits of s to overwrite, the very policy Infini-attention lacked.

Constant state buys efficiency at the cost of exact recall, and deployed systems respond by keeping a thin layer of softmax attention for precise short-range access while delegating the long tail to cheap recurrent state, a multi-fidelity (P-fid) split across layers. Jamba interleaves Transformer and Mamba blocks with a mixture-of-experts feed-forward [64]; Samba alternates Mamba layers with sliding-window attention [96]; Griffin mixes gated linear recurrences with local attention [25]. Each spends a small precise budget where queries need random access and a large lossy budget where they need only gist, an architectural realization of the prediction that query-relevant bits should be held at high fidelity and the rest at low.

A further strand makes the write policy itself adaptive at inference. Titans makes the recurrent memory a deep module whose weights are updated at test time by gradient descent on a “surprise” (memory-prediction-error) signal with momentum and adaptive forgetting, scaling to multi-million-token contexts [6]. This is a learned C_θ that re-optimizes its own distortion surrogate online. Larimar couples a decoder with a Kanerva-style distributed associative memory, enabling one-shot, training-free writes, edits, and selective forgetting at inference, at the cost of a lossy associative read bounded by the memory dimension [24]. MemoryLLM embeds a fixed-size pool of latent memory tokens in every layer that self-updates as new text arrives, with controlled forgetting [114]. Its pool is again a fixed B , and retention degrades once history exceeds it, the bound observed empirically.

Two methods convert an existing model into a bounded-state one without retraining from scratch. Dynamic Memory Compression (DMC) learns, per head and via continued pretraining, whether to append a new key–value pair or accumulate it into the previous slot, retrofitting an adaptive, content-aware compression rate onto a standard cache [82]. TransMLA converts a pretrained grouped-query-attention model into a multi-head latent attention model [28, 75], replacing the GQA cache with the low-rank latent state of MLA. This swaps the currency of $\text{rate}(Z)$ after the fact, bridging this section’s trained-from-scratch end to the inference-time factorizations of Section 4.

Constant memory versus exact recall. Every mechanism here runs into the same constraint. A fixed s -bit state cannot answer queries whose answers require more than s bits, and no write policy escapes the data-processing inequality behind Eq. (2). The surrogate that decides which bits to keep can improve; the ceiling cannot. Nor is the bound merely asymptotic. RNNs with bounded state are provably weaker than transformers at in-context retrieval [117], and RMT, precisely because its memory size is an explicit knob, finds needles in an 11M-token haystack that full-attention models miss, by spending its small budget on the relevant span and discarding the rest [54, 55]. That result is the bound made positive: when $I^*(Q)$ is small and the operator is query-aligned, a tiny s suffices; when the query distribution is broad or answers carry many bits, no fixed s does. So the architectural strand chooses, at training time, where on the rate–distortion frontier to sit, rather than evading the frontier altogether.

7 Agent and Semantic-Memory Compaction

This is the longest-timescale layer, and the one where the formalism’s abstractions turn into concrete operations. The history H here is the accumulated trace of an agent’s life: tool calls, observations, dialogue turns, and prior reflections, often spread across many sessions, not a KV cache or a token window. The compact representation Z is an external, mostly natural-language store of summaries, extracted facts, structured notes, or a knowledge graph, and the usage operator $\mathcal{U}$ is a retrieval step that pages a subset of Z back into the window before generation. Two features set the layer apart. The budget B is enforced not by hardware but by the context window of $\mathcal{U}$: an unbounded store is useless if the agent can only read a few thousand tokens of it, so the bottleneck is retrieval, not storage. And the distortion surrogate is almost always *LLM-judged salience*. An LLM scores, summarizes, or decides what to keep, so the compaction operator is itself a fallible model-in-the-loop rather than a cheap statistic. This is expensive in two ways: every compaction event is an LLM call, so the savings a smaller context buys are partly repaid in inference overhead, and any error the judge commits is written into Z for good. The decisive axis here is reversibility (P-rev). Schemes that keep the raw trace archived and retrievable are reversible; schemes that overwrite it with a lossy summary are not, and the irreversible ones drive prediction (4).

The first family compacts the running trajectory of a single task as it approaches the window limit. MemGPT/Letta casts this as operating-system paging: the LLM treats its window as “main memory” and an external vector store as “disk,” and issues function calls to evict, summarize, and page content back in [86]. The OS metaphor is exactly the rate–budget trade of Eq. (1), with B the window and the store unbounded but slow, and its weakness is the one the formalism names: paging is governed by the model’s own summaries, so eviction is irreversible whenever the archival copy is itself a summary rather than the raw turn. ACON keeps the operator training-free but makes the surrogate explicit. It compresses observations and history under natural-language “compression guidelines” that are iteratively refined from paired trajectories where the full context succeeds but the compressed context fails, a failure-driven estimate of which dropped bits raised distortion [53]. The 2025–2026 frontier replaces hand-tuned operators with RL-learned compaction policies that optimize a proxy for $\mathcal{D}(\theta)$ directly. Context-Folding branches into a sub-trajectory for a sub-task and, on return, folds away the intermediate steps, keeping only an outcome summary, trained end-to-end with the FoldGRPO objective [105]; Memory-as-Action frames in-place edits (prune-and-write deletions and insertions) as policy actions learned jointly with task-solving [141]; MEM1 carries a single constant-size internal state that it rewrites every turn by merging prior memory with the new observation and discarding redundancy [148]; and ReSum periodically invokes a summarization tool to collapse a web-search history into a compact reasoning state, with ReSum-GRPO training the agent to reason fluently from summaries [120]. Iter-Research rebuilds a streamlined workspace each round from only the essential prior outputs, treating an evolving central report as the agent’s Markovian memory [15]. These methods report 3–10× active-context reduction at matched accuracy. They also share a structural risk: all fold, prune, and rewrite irreversibly, so a detail

the policy judged unimportant is gone, and the RL signal optimizes average distortion, not the tail-query distortion that the bound shows is where low-budget memory fails.

The second family compacts across task boundaries, promoting raw episodes into reusable knowledge. Its canonical pipeline is reflection. Generative Agents keep a timestamped memory stream scored by recency, importance, and relevance, and periodically synthesize salient memories into higher-level reflections that are themselves stored and retrieved [88]; the importance score is the prototypical LLM-judged salience surrogate. Reflexion specializes this to failure, writing a verbal self-critique after a failed attempt and prepending it on the next trial [101], and ExpeL distills cross-task experience by contrasting successful and failed trajectories into natural-language insights [143]. Being gradient-free makes these methods cheap and general, but it also exposes the layer’s signature pathology: a wrong reflection or over-generalized rule is stored and then biases every future retrieval. This *self-reinforcing error* is the agent-level analogue of an evicted needle, with a twist: the agent does not just lose a good memory, it reconstructs a bad one on demand. A second sub-family treats consolidation as structured editing. Mem0 runs an extract-then-update pipeline in which an LLM pulls salient facts and then issues ADD/UPDATE/DELETE/NOOP operations to keep a compact, consistent store [21]; A-MEM builds Zettelkasten-style atomic notes that are dynamically linked to and evolve related notes into a self-organizing network [125]; Memory-Bank adds explicit forgetting, decaying memories by an Ebbinghaus curve so that significance and recency, not capacity alone, govern what survives [146]; and MIRIX routes information into six typed modules (core, episodic, semantic, procedural, resource, knowledge-vault), each with its own fields and access policy [113]. The UPDATE/DELETE operator is the cleanest realization of an irreversible C : a mis-merge or wrongful deletion cannot be re-derived, so its error is permanent in exactly the sense (P-rev) isolates. Sleep-time compute reframes the economics, moving consolidation offline so that the LLM “thinks” about a context before any query arrives and caches useful inferences, amortizing the per-compaction LLM-call cost across future queries [65]. The catch is that it spends budget speculatively on bits that may never be queried, the dual of the query-agnostic penalty $H(Q)$ in Eq. (2).

A third family makes the store itself structured, so that $\mathcal{U}$ retrieves abstractions rather than raw chunks and compaction moves into an offline indexing stage. RAPTOR recursively clusters and summarizes chunks into a bottom-up tree, letting retrieval draw from multiple abstraction levels, a multi-fidelity (P-fid) store with lossy summaries above lossless leaves [97]. GraphRAG extracts an entity graph, detects communities, and pre-generates community summaries for global queries [32]. HippoRAG builds an open knowledge graph and runs Personalized PageRank over it for single-step multi-hop retrieval, with HippoRAG 2 deepening passage integration toward non-parametric continual learning [42, 43]. Zep maintains a temporal knowledge graph, time-stamping and versioning facts so that updates and contradictions are reconciled by recency rather than overwritten [95]. Of the family it preserves reversibility best: a superseded fact is retired, not deleted. The parametric counterpart edits weights directly. ROME locates and rewrites the single mid-layer MLP module that mediates a factual association via a rank-one update, and MEMIT scales this to thousands of edits

spread across a range of layers at once [76, 77]. Parametric editing eliminates retrieval latency, since $\mathcal{U}$ becomes a forward pass, but it is maximally irreversible and hard to audit, and at scale it induces the catastrophic forgetting that is the parametric analogue of summary collapse.

EM-LLM collapses the distance between this layer and the KV layer of Section 4 [35]. It segments the token stream into events by surprise, stores each event’s KV block as an episodic memory unit, and at decode time retrieves relevant events by similarity and temporal contiguity, a retrieval over a memory store whose items happen to be KV blocks rather than text notes. It is at once a KV-cache method (the substrate is on-device key-value tensors) and an agent-memory method (the operator is event segmentation plus salience retrieval), which is why it tests the bridge thesis empirically: the same reversible, retrieval-backed design that keeps an agent’s note store from collapsing keeps an episodic KV memory from collapsing, because both obey Eq. (2) with the same $I^*(Q)$.

Persistent, self-written, and trusted on later retrieval, Z is also an attack surface. AgentPoison backdoors an agent by poisoning its long-term memory or RAG store so that a trigger in a future query retrieves the malicious entry and steers the agent’s action [18]. MINJA injects a corrupted memory through query-only interaction, with no privileged access, exploiting the very write-on-experience loop that consolidation relies on [30]. These are the adversarial form of the self-reinforcing-error problem: a single planted memory, like a single bad reflection, biases all subsequent retrieval, and reversibility offers little defense once the poisoned item is the one retrieved.

Reversibility and the compounding of loss. Across all four families the distortion surrogate is LLM-judged salience, and (P-rev) is the axis that decides their fate. Reversible schemes (archival paging with raw copies, temporal graphs, multi-fidelity trees) can re-derive a dropped detail when a later query needs it, so their error stays near the budget-limited floor of Eq. (2). Irreversible schemes (lossy summarization, fold/prune, UPDATE/DELETE, weight edits) cannot, and applied repeatedly, each compaction event composes its loss with the last. This is prediction (4): under repeated irreversible summarization, end-task error should grow super-linearly in the number of compaction events, because errors both accumulate and self-reinforce (a stored mistake biases the retrieval that feeds the next summary), whereas a reversible, retrieval-backed memory stays flat. It is also the layer’s least-measured failure. Benchmarks such as LOCOMO and LongMemEval probe single-shot recall, multi-session reasoning, and knowledge updates, but not the compounding curve over many consolidation cycles [74, 118]. That gap is why we test it directly in Section 14, and why recent surveys of agent memory and its security flag lossy consolidation and memory integrity as the open problems [47, 66].

8 Trainable Sparse Attention

The KV-cache methods of Section 4 share a structural limitation that the formalism names precisely. They are training-free heuristics that fix a compaction operator C after the model is trained, then ask the frozen weights to tolerate it. On the learnability axis (Section 3) they sit at the “training-free” end; on the query-adaptivity axis most sit at “query-agnostic.” That double placement is the exact

corner the lower bound penalizes. An evictor that decides what to keep before seeing the decode-time query pays the $H(Q)$ price of consequence (iii) in Section 2, and a heuristic surrogate (accumulated attention mass, recency) is only a coarse estimator of the true distortion. The methods here attack both deficits at once, making C itself learned and end-to-end: the sparsity pattern is a parameter the model is trained around, so the weights and the retain/discard rule co-adapt instead of fighting. This is the learnability $\times$ query-conditioning corner that no frozen-model evictor can reach, and where the 2025 production frontier has settled, spanning both the KV layer and the architecture itself.

Natively trainable sparse attention. These methods train with sparse attention from the start rather than retrofit it. Native Sparse Attention (NSA) [135] composes three branches whose outputs are gated and summed: coarse-grained compression of token blocks into block-level summaries, fine-grained selection of the most relevant blocks for full attention, and a sliding window for local context. The selection is differentiable and the kernels are hardware-aligned to GPU tensor cores, so the scheme is both trainable end-to-end and fast on real hardware, matching or beating full attention while reading a fraction of the cache. In our terms NSA is a learned, query-conditioned C_θ whose compression branch supplies a low-rate global sketch and whose selection branch supplies a high-fidelity tier, a multi-fidelity profile (P-fid) the trained weights learn to exploit. MoBA [73] reaches the same corner through MoE-style routing: a learned gating function routes each query to a sparse subset of key–value blocks, so a query attends only to the blocks a trained router deems relevant, and the model can switch between sparse and full attention without retraining. The retain decision is thus query-conditioned by construction, and the router is the query-aware selector that Quest approximates heuristically. DeepSeek Sparse Attention (DSA) [27], shipped in DeepSeek-V3.2, makes the frontier concrete: a lightweight “lightning indexer” scores which past tokens each query should attend to, and fine-grained selection over those tokens cuts long-context cost in a deployed model. DSA shows that learned sparse attention has crossed from research into production infrastructure.

A complementary line keeps the model weights fixed and instead learns, or calibrates offline, the shape of the sparsity, exploiting the fact that attention maps are structured. MInference [48] finds that long-context prefill attention falls into a few recurring patterns (A-shape, vertical-slash, and block-sparse), assigns each head its pattern offline, then computes only the dynamically located nonzero regions at inference, slashing the quadratic prefill cost with little quality loss. This is a query-conditioned C acting at the prefill stage of the lifecycle axis, the one stage the frozen decode-time evictors of Section 4 leave untouched. Even without retraining, then, recognizing the learned structure of attention recovers much of what a query-agnostic heuristic discards.

Attention heads are not interchangeable: a minority of “retrieval heads” carry the long-range copying that needles depend on, while the rest are effectively local. DuoAttention [122] learns a binary label per head with a lightweight optimization. Retrieval heads keep the full KV cache; streaming heads keep only attention sinks plus a recent window. The expensive full-fidelity budget thus goes only where the bound says it must. RazorAttention [107] reaches a

similar split training-free, identifying retrieval heads by their attention structure and compressing the rest while protecting dropped information with a compensation token. Both instantiate prediction (1): full fidelity is preserved on exactly the heads whose evicted tokens would otherwise cause needle recall to collapse, and budget is reclaimed from heads whose answer-relevant information content I^* is low. They differ on the learnability axis, DuoAttention learning the label and RazorAttention inferring it, but both allocate the same fidelity budget at head granularity.

SeerAttention [37] makes the selection rule an explicit learnable component: a self-distilled gate learns each block’s attention sparsity intrinsic to the model, turning the implicit sparsity that evictors guess at into a trained, queryable signal. Two other methods make the budget structure-aware. SCOPE [119] observes that prefill and decode have different compaction needs, since decoding accumulates its own heavy hitters, and allocates separate KV budgets to the two phases with a sliding decode strategy, dropping the assumption built into decode-time evictors that one budget fits both stages of the lifecycle. SepLLM [16] exploits a structural regularity: separator tokens absorb disproportionate attention mass, so a segment’s information can be compressed into its trailing separator, a structural token-level summarization that applies training-free and can also be folded into training.

These methods answer the query-agnostic failure diagnosed in Section 4 head-on. By folding C into the trained model, NSA, MoBA, and DSA make the retain/discard decision query-conditioned and co-adapted with the weights, sidestepping the $H(Q)$ penalty instead of paying it. MInference recovers the prefill stage, DuoAttention and RazorAttention spend fidelity by head as the bound prescribes, and SeerAttention learns the gate, while SCOPE makes the budget phase-aware and SepLLM the summary structure-aware. The cost is the learnability itself: a frozen evictor is plug-and-play, whereas natively sparse attention demands training or distillation and so binds the operator to a model family. The direction of travel is plain. The compaction operator is migrating out of the post-hoc heuristic layer and into the model, which is why these methods, unlike those of Section 4, are the ones that reach production.

9 Multimodal and Multi-Agent Compaction

Two settings break the assumption that the unit of compaction is a text token in a single model’s context. A multimodal model’s history H is dominated by visual tokens; a multi-agent system’s history is a collection of inter-agent messages spread across collaborators. Both fight the same quadratic-attention bottleneck, yet they sit at very different points of the rate–distortion picture of Section 2, because their redundancy structures differ. The formalism predicts where. At fixed quality the achievable budget scales with the task-conditioned information content $I^*(Q)$, and a modality whose tokens are mutually predictable carries a small $I^*(Q)$ per token, so it survives much harder compression than text before the bound bites.

Compacting the visual stream. A single image expands into hundreds of patch tokens whose neighbors are spatially correlated, and video multiplies this again across near-duplicate frames. The mutual information $I(Z; H)$ needed to preserve answer-relevant content is therefore a small fraction of the raw token count, and

in practice an 8–18× reduction costs almost nothing. The earliest responses were architectural. A Perceiver Resampler lets a small set of learnable latent queries cross-attend over variable-length image or video features and distill them into a fixed budget of, say, 64 visual tokens [2]. This is a learned C with a hard, query-agnostic rate cap. It decouples LLM cost from input resolution, but because it ignores the query it pays the $H(Q)$ penalty of Eq. (1) and caps fine-grained detail on dense or OCR tasks. Training-free methods soon exploited the same redundancy with no retraining. Token Merging (ToMe) gradually merges the most similar tokens in each block via bipartite soft-matching on key cosine similarity [7]; its merge operator is irreversible (P-rev) and its distortion surrogate is pure embedding similarity, so it is task-agnostic and tends to over-merge distinct foreground. FastV keeps full attention in the first two LLM layers and then prunes low-attention visual tokens in deeper layers [17], exploiting that visual tokens turn redundant after early layers. Its surrogate is text-to-visual attention mass, which later work showed misaligns with true visual importance, exactly as the bound’s importance-heuristic analysis predicts. VisionZip moves the compression ahead of the LLM, selecting dominant tokens by encoder attention and merging the rest into a small contextual set [127]. LLaVA-PruMerge instead reads CLS-to-patch attention sparsity through an IQR outlier criterion, prunes adaptively, and re-merges the pruned tokens back into the survivors to reach $\sim 18\times$ compression [99]. ToMe, VisionZip, and LLaVA-PruMerge are query-agnostic (P-q absent) and irreversible (P-rev absent); FastV is only weakly query-conditioned through its text-to-visual attention surrogate but is likewise irreversible. The bound pins their shared failure mode: evidence for a later, unanticipated grounding query is merged away once and cannot be recovered, which is just the detail-sensitive regime where these methods break down.

A second line carries the KV-cache machinery of Section 4 over to the mixed-modality cache. Its governing fact is that the two modalities have different I^* per token, which makes a uniform budget the wrong default. LOOK-M evicts and merges KV entries under a text-prioritized policy, protecting textual keys while merging the redundant visual ones [112]; this encodes the observation that text tokens carry more answer-relevant bits per token than image tokens do. VL-Cache turns the same asymmetry into explicit budget allocation, distributing a global cache budget across layers and modalities according to their measured sparsity [110]. A modality-aware B split is the right move whenever one stream’s tokens are far more compressible than the other’s.

A fixed cache is hopeless once the stream runs to 10K+ frames, so systems tier short- and long-term memory, an architectural analogue of the agent-memory consolidation of Section 7. MovieChat fills a FIFO short-term buffer with dense frame tokens and, on overflow, merges adjacent similar embeddings into a compact long-term memory [103]. Its Atkinson–Shiffrin-style two-tier C consolidates by similarity and ignores the question, so query-specific evidence can vanish on overflow. MC-ViT takes a different route, fine-tuning a pretrained video transformer to attend to a non-parametric memory bank of redundancy-reduced past-segment activations [4]; this extends temporal context without architectural change, but the memory still grows with video length. Both are multi-fidelity in spirit, pairing a dense recent tier with a consolidated distant tier

(P-fid), and both stay irreversible. The bound warns that without query-conditioning the consolidation step discards bits the future may need, and needle-in-a-long-video recall is indeed where these methods are weakest.

When the history is spread across collaborating agents, the unit becomes the inter-agent message, and the redundancy is discrete-semantic, repeated reasoning and restated subgoals rather than continuous spatial structure. Compaction turns abstractive here: it summarizes transcripts into reusable insights, and it carries provenance and access control that single-context compaction never has to. G-Memory builds a hierarchical three-tier graph, an interaction-transcript graph, a query graph, and an insight graph, that distills raw inter-agent dialogue into condensed trajectories and abstract, reusable insights retrieved by bi-directional traversal [136]. In the rate–distortion view this is a lossy semantic C over messages whose distortion surrogate is the LLM’s own salience judgment, as in Section 7, with one extra constraint: an edge encodes which agent may read which fragment, so the budget is shared across principals instead of owned by one.

The two threads barely touch. On the visual side the work compresses continuous, spatially redundant embeddings, almost always training-free and similarity-based; on the multi-agent side it abstracts over discrete messages under governance. Their intersection has no information-theoretic account: compacting multimodal context shared across collaborating agents, a blackboard holding compressed video memory that several agents read under different access rights. The bound of Eq. (2) is stated for a single Z answering a single Q , and it says nothing yet about how to size a shared B when agents draw different $I^*(Q)$ from one compressed multimodal store, or how irreversible visual merging collides with the provenance a downstream agent may later need. We still lack a theory that quantifies how much visual and temporal redundancy is compressible without task loss, set against the discrete-semantic redundancy of messages, and such a theory is what would ground the robust empirical fact that visual tokens compress far harder than text.

10 The Inference $\leftrightarrow$ Agent-Memory Bridge

The four communities surveyed so far act on different substrates at different timescales, yet they are working the same design problem. That problem shows up at three tiers of a single memory hierarchy, and because it is one problem, the mechanisms each community has refined carry across tiers rather than staying put. This section makes the claim concrete: it aligns the tiers, traces the same five design knobs through each of them, ports specific mechanisms across the bridge, marks where the analogy breaks down, and ends with five compaction-aware design principles the formalism implies.

10.1 One hierarchy, three tiers

A modern agent keeps memory at three scales that the literature usually studies apart. The **KV cache** operates sub-second, on-GPU, and near-lossless at the granularity of a token: it holds the attention state of the current forward pass, and compaction here (Section 4) decides which tokens’ keys and values survive within a single generation. The **working context** lives inside the context window for the span of a task: it holds the agent’s running trajectory, and

Design knob	KV cache	Agent memory	long-term
Importance signal	attention [60, 142]	score	LLM salience [21, 88]
Forgetting	window / eviction [123]	decay curve [146]	
Query-cond. (P-q)	per-query [108]	pages	recall-time retrieval [97]
Reversibility (P-rev)	keep-all vs. evict [108, 142]	archive [86]	vs. summarize
Budget alloc.	per-head/layer [11, 34]	per-memory-type [113]	
Stop rule	output-error bound [34]	(open)	

Table 4: The bridge. The same design knobs recur across the memory hierarchy; a mechanism refined at one tier is a candidate design for another, and an empty cell (a principled stop rule for agent summarization) is a research opportunity the alignment makes visible.

compaction here (Section 7) decides which observations, tool results, and reasoning steps stay visible as the task unfolds. The **long-term store** is external and semantic and persists across sessions: it holds what the agent carries from one task to the next, and compaction here decides which facts, summaries, and episodes get written, kept, and retrieved. Their timescales span many orders of magnitude, but all three answer the same Eq. (1): keep the information a future query will need, under a budget, at minimal distortion.

10.2 The same five knobs

Since the three tiers share one objective, the same design knobs turn up at each, and a knob one community has studied in depth is often blank in another (Table 4). The importance signal is attention score in the KV cache [142] and LLM-judged salience in the store [21]; both are surrogates for the same distortion gradient (Section 2). Forgetting takes the form of a sliding window or eviction schedule in the cache [123] and an explicit decay curve in agent memory [146]. Query-conditioning divides methods that keep all state and select per query (Quest; 108) from those that commit in advance, with retrieval at recall time versus eager summarization as the agent analogue. Reversibility runs from keep-all paging to permanent discard at every tier. The last knob, a calibrated stop rule for when compaction has gone too far, exists at the KV layer as an output-error bound [34] and is all but absent for agent summarization, a gap the bridge brings into view.

10.3 Mechanisms that port across the bridge

Once the knobs are read as shared, each column becomes a source of designs for the others. Three ports show the payoff. Memory-Bank’s Ebbinghaus-style forgetting curve [146], a recency- and reinforcement-weighted decay for cross-session memory, is the content-aware prior KV eviction has been missing: H2O and its successors evict by accumulated attention alone [142], a signal

biased toward early tokens, whereas a decay prior calibrated to re-access statistics drops straight in as a scoring alternative. Quest’s discipline of never evicting, and instead selecting query-relevant pages on demand [108], gives agent memory a template: an agent that retrieves from an archival store at recall time (reversible and query-conditioned) dominates one that eagerly summarizes, on exactly the queries the summary dropped. Ada-KV’s output-error bound [34], which caps the deviation a given eviction induces in the attention output, supplies the stop rule agent summarization lacks; instead of compacting on a fixed schedule, an agent could compact until a bounded estimate of induced output error is hit. EM-LLM [35], which segments the KV cache into episodic events and retrieves them like memories, already sits on the bridge and is the cleanest empirical sign that the two tiers run on one logic.

10.4 Where the analogy breaks

The analogy has real limits, and they are worth stating. Recall guarantees differ across the tiers: a KV cache that keeps a token’s entries can attend to it exactly, while a store that keeps a summary of an episode has already paid an irreversible distortion. The meaning of “query” differs too. At the KV tier it is the next-token computation, implicit in the current hidden state; at the agent tier it is a future user request, genuinely unknown and possibly adversarial. Cost is the third difference. KV compaction is a microsecond arithmetic decision, whereas agent consolidation is itself an expensive, error-prone LLM call, so the agent tier pays a budget the cache does not: the cost of deciding what to keep. None of this refutes the bridge; it refines it, telling us which ports are clean (scoring signals, reversibility disciplines) and which need adaptation (anything that assumes free or exact recall).

10.5 Five compaction-aware design principles

Between them, the formalism and the bridge imply five principles for systems that compact memory well. Each comes with supporting evidence and a failure the formalism predicts when the principle is ignored.

P1. Never discard irreversibly what you cannot re-derive cheaply. At equal budget a reversible operator (retrieval-backed eviction, archival memory) weakly dominates an irreversible one (permanent eviction, lossy summarization): the two can differ only on queries whose evidence was dropped, and there the reversible one wins. Quest [108] follows P1; H2O [142] and fixed-schedule summarization violate it, with the predicted failures of recall collapse (prediction 1) and error accumulation (prediction 4).

P2. Condition on the query distribution, not just the current query. The bound charges a query-agnostic operator the entropy $H(Q)$ (Section 2). Trainable sparse attention (Section 8) and query-aware compression [50] pay less of this penalty; offline gisting pays it in full.

P3. Separate a cheap reversible episodic tier from a lossy semantic tier, with explicit promotion and demotion. Multi-fidelity memory (P-fid) dominates a single lossy tier: keep raw episodes cheaply and recoverably, and abstract only what proves reusable. MIRIX’s typed modules [113] and the Compressive Transformer’s dual tier [93] are built this way; a flat summarization buffer is not.

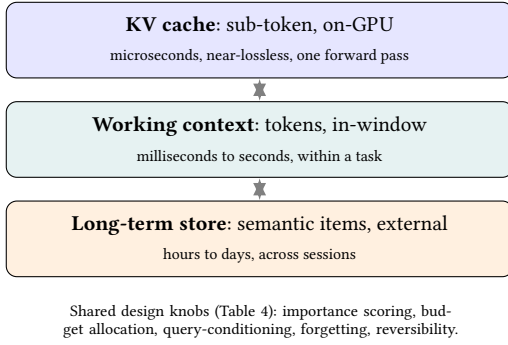


Figure 4: The memory hierarchy of Section 10. Three tiers separated by many orders of magnitude in timescale share the same five design knobs, so a mechanism refined at one tier becomes a candidate design for the others.

P4. Make compaction asynchronous and learnable, not a blocking heuristic. When deciding what to keep is itself an LLM call on the critical path, it erodes the savings it buys. RL-learned policies [105, 141] and sleep-time consolidation [65] move the decision off the critical path and learn it; fixed-threshold auto-compaction does neither.

P5. Budget by marginal task-utility, not a uniform ratio. The bound is governed by $I^*(Q)$, which spreads unevenly across heads, layers, and memory items. Non-uniform allocation [11, 34] respects this; a global fixed ratio ignores it and spends budget on low-utility state.

11 Systems and Serving Substrate

The preceding sections treated compaction as an algorithm. This one treats it as a resource policy. At the serving runtime, the lifecycle stage in our taxonomy (Section 3) where memory is managed across requests rather than within one forward pass, the KV cache stops being a per-sequence buffer and becomes a shared, multi-tier asset whose allocation, reuse, and movement a scheduler decides. The same rate-distortion objective (Eq. 1) still governs, but the currency of $\text{rate}(Z)$ is now GPU high-bandwidth memory weighed against bus bandwidth and latency SLOs, and C lives in a memory manager rather than an attention kernel. Read this way, serving systems split into two families that the lens places at opposite ends of the (P-rev) axis, and the interaction between them is the central tension of the stack.

The first family does not compress at all; it retains and shares exact KV. PagedAttention [56] ports OS virtual-memory paging to the cache, storing it in fixed-size non-contiguous blocks behind a per-sequence block table. This eliminates fragmentation and, decisively for our argument, enables copy-on-write sharing of identical prefixes across requests. RadixAttention in SGLang [145] generalizes the idea to a radix tree keyed by token sequence, so an incoming request automatically reuses the longest cached prefix: system prompts, few-shot exemplars, and branching reasoning trees are computed once and read many times. CachedAttention [36] stretches the horizon across conversational turns, spilling KV to

DRAM and SSD tiers with layer-wise prefetch to hide slow-tier latency. vAttention [91] questions paging itself, using CUDA virtual-memory APIs to keep the cache contiguous, so unmodified kernels run, while still allocating physical pages on demand. In our formalism these are the limiting case of zero distortion: a cached prefix is perfectly reversible memory, C approaching the identity, the opposite extreme from eviction. Their leverage comes not from spending fewer bits but from amortizing bits already spent, which is why the gains are entirely workload-dependent and collapse to nothing without prefix overlap. Reuse, unlike compression, buys nothing when there is nothing to reuse.

A second runtime concern is not how large the cache is but where it lives. Prefill is compute-bound and decode is memory-bound, so co-locating them lets each interfere with the other’s SLO. Splitwise [89], DistServe [147], and Mooncake [92] disaggregate the two phases onto separate machines, which turns the KV cache into a tensor that must be transferred between prefill and decode pools over the interconnect. DistServe places the phases by cluster bandwidth to bound transfer cost while optimizing TTFT and TPOT independently for goodput. Mooncake goes further and pools idle CPU, DRAM, and SSD across the cluster into a disaggregated KVCache store driven by a cache-centric scheduler. Here $\text{rate}(Z)$ returns not as an occupancy budget but as a bandwidth budget: the cost of an exact, reversible representation is the time to ship it, which is what makes the lossy and the transmissible families below attractive even when memory is not scarce.

The third family reintroduces distortion to fit large models and long contexts into a fixed hierarchy. FlexGen [100] aggregates GPU, CPU, and disk, solves a linear program for tensor placement, and quantizes weights and the cache to four bits, trading precision and interactivity for capacity in offline batch settings. InfiniGen [57] keeps only a working set on-GPU, rehearsing the next layer with partial weights to guess which few tokens will matter and prefetching only their KV from host memory. ShadowKV [104] offloads the value cache, retains a low-rank key cache plus landmarks and outliers, and reconstructs keys to fetch only the values it needs. These are the query-conditioned, approximate operators of Section 4, now embedded in the memory manager: their selection is lossy (P-fid) and their dropped entries are not re-derivable within the request (irreversible, $\neg$ P-rev).

A bridging family makes exact KV cheap to move rather than cheap to store. CacheGen [70] encodes the cache into a compact bitstream that exploits its layer- and token-locality and streams it, trading a controlled, near-lossless quantization error for far lower transfer time, which puts compaction in the service of disaggregation. CacheBlend [131] attacks a limitation all of the above share, that they reuse only prefixes: in RAG the relevant chunks arrive concatenated in query-dependent order, so prefix matching fails. CacheBlend reuses each chunk’s independently precomputed KV and recomputes only the small fraction of cross-attention entries that the chunks’ new neighbors change, recovering most of the quality of full attention at a fraction of the prefill cost. This is a multi-fidelity operator (P-fid) in the wild: a large reused tier plus a tiny recomputed tier sized by an error budget.

The lens exposes a tension these systems must resolve jointly. Lossless reuse (PagedAttention, RadixAttention, prefix caches) assumes the stored KV is exactly what attention would have produced; lossy budgeting (InfiniGen, ShadowKV, quantized offload) assumes it may be approximated. Composing the two operators costs something: reusing a prefix that was itself evicted or quantized propagates the earlier distortion into every request that shares it, and the errors compound across the reuse tree instead of staying local to one sequence. With the two families at opposite ends of (P-rev), the open systems problem is a scheduling question over one budget. When should KV stay reversible and exact, paying storage and bandwidth, and when should it go lossy and cheap, paying distortion? And how does one stop a lossy decision made for a single request from silently degrading every descendant that later reuses it?

12 Theory of Compaction

The formalism of Section 2 reduces compaction to a single rate–distortion program (1) with a single lower bound (2). This section collects the rigorous results that either instantiate that bound or sharpen the constant in front of it, and closes on the one quantity the theory still cannot predict: $I^*(Q)$ itself.

The cleanest instance of the bound is a space lower bound on lossless KV caching. Haris and Onak [44] prove that any algorithm performing exact attention-based generation over a length- n , dimension- d context must use $\Theta(nd)$ space, with a matching $\Omega(d \cdot e^d)$ barrier in the low-dimensional regime and no non-adaptive sublinear-time online attention. Read through (2), this says exact attention demands $B \geq I^*(Q)$ for the worst-case query: when the task is verbatim recall over an incompressible context, $I^*(Q)$ is itself $\Theta(nd)$, so no Z smaller than the cache can drive P_e to zero. Sub-linear caches are therefore impossible without structural assumptions on H , the very assumptions our $1/I^*(Q)$ compression ratio exploits whenever real data is redundant. The same shape recurs for fixed-state recurrence. Wen et al. [117] show that fixed-size RNN/SSM states provably cannot solve the in-context associative recall or tree recognition that attention solves, that chain-of-thought alone does not close the gap, and that a single attention layer (or RAG) restores it. This is consequence (iii) of our bound made architectural: a state of B bits cannot answer a query needing more than B bits, so the separation turns on exact in-context retrieval, the regime where $I^*(Q)$ is large and query-agnostic ($H(Q)$ -blind) compaction pays most. Parametric memory completes the picture. Nichani et al. [83] characterize it directly: transformers store facts as associative memories whose capacity scales linearly in parameter count, with a single attention-plus-MLP block near-optimal. This puts a concrete I^* -budget on the weights rather than the cache, and it explains why some history need never be retained as Z at all, since facts already amortized into parameters contribute zero to $I(Y; H | Q)$.

The bound says how many bits must survive; attention sinks say which tokens hold the survival machinery. Xiao et al. [123] observed that softmax forces large attention mass onto the first few tokens irrespective of content, and that retaining ~ 4 sink tokens plus a sliding window streams to millions of tokens. Gu et al. [41] show empirically that sinks are universal, emerge only after

enough optimization, act as “key biases” that absorb surplus attention without contributing to value computation, and vanish under un-normalized (sigmoid) attention. A sink is therefore an artifact of the softmax normalizer, not of $I^*(Q)$. The decisive result for compaction is Ran-Milo [94]: for a natural family of trigger-conditional tasks, any softmax model that solves them must develop a sink, whatever its training. Sinks then carry irreducible $I^*(Q)$ for those tasks, so evicting them is not lossy compression but a violation of budget feasibility in (1), and the failure is structural rather than graceful. de Llano et al. [26] unify sinks with depth-localized “compression valleys” (sharp drops in residual-stream rank) as two faces of the same massive-activation mechanism concentrated in middle layers. On the representation side this predicts where H is most compressible across depth, and it warns that the high-fidelity tier of any (P-fid) profile must go to the anchor tokens the valleys form around.

The bound is also location-dependent. Liu et al. [68] document a U-shaped recall curve: accuracy is highest when the evidence sits at the context’s start or end and collapses in the middle. In our terms this is a distortion $\mathcal{D}(\theta)$ that depends on where the answer-bits sit within H , even at fixed B and fixed $I^*(Q)$. It sharpens prediction (1) of the formalism, since the eviction collapse point tracks the entropy of the answer’s location, so a compaction scheme uniform in position silently raises P_e for mid-context evidence. Placement, not retention alone, is a free variable in the rate–distortion program.

The objective (1) rests on the rate–distortion/IB problem of Tishby et al. [109]: trade $I(Z; H)$ against $I(Z; Y | Q)$, keeping the answer-predictive bits and spending nothing on the rest. Wang et al. [115] make this operational for context compression, recasting token selection as maximizing mutual information with the query/answer under a token budget and using cross-attention as the relevance estimator. This is the query-conditioned (P-q) reading of our surrogate-distortion view: it estimates $I(Z; Y | Q)$ directly rather than through the perplexity proxy, which is why an IB objective beats self-information pruning by roughly the mutual-information gap of prediction (2).

Every result above is worst-case or mechanistic, and none predicts the 80–93% KV reductions and $\sim 20\times$ prompt compression that practice routinely achieves, with quantization reportedly within $\sim 2.7\times$ of the information-theoretic limit. The reconciliation is consequence (ii). Worst-case $I^*(Q)$ is $\Theta(nd)$, but on realistic data $I^*(Q)$ is far smaller, and the achievable ratio is $\propto 1/I^*(Q)$. The gap between the lower bounds and observed compressibility is thus the gap between worst-case and data-conditional $I^*(Q)$, and that gap is not yet characterized. What the theory still owes us is a predictive compression scaling law: a closed-form estimate of $I^*(Q)$, and hence of the achievable ratio at fixed quality, as a function of model size, context length, head/layer structure, and task type. The results we have, linear parametric capacity, near-limit quantization, sink necessity, U-shaped recall, are scattered estimators of pieces of $I^*(Q)$. Unifying them into one law that predicts, before compaction runs, how much memory a given model on a given task can shed is the central unsolved problem this survey leaves open.

13 Evaluation and the COMPACT-Bench Proposal

Whatever a compaction method claims, the claim only holds up to the evaluation that backs it. The formalism of Section 2 pins down three things such an evaluation has to measure: the accuracy reachable at a fixed memory rate B ; whether a gap between two schemes at equal B traces back to reversibility (P-rev) and query-conditioning (P-q); and whether the four predictions hold layer by layer. No current benchmark measures these directly. Below we review the suites that exist, pin down what they miss, and sketch a benchmark built to fill it.

13.1 What exists

The field started from one synthetic probe and grew along three axes. Synthetic stress tests buy controllable length and difficulty. Needle-in-a-Haystack drops a single fact at varying depth and plots a depth $\times$ length heatmap [51]. RULER widens this to thirteen multi-key, multi-hop, and aggregation tasks, finding that barely half of the models advertising $\geq 32K$ actually survive at 32K [46]. BABILong buries bAbI reasoning facts in book-length filler and reports that models exploit only 10–20% of their stated window [54]. Each of these has a natural reading in terms of $I^*(Q)$: single-fact NIAH carries little information and saturates, whereas the multi-hop demands of RULER and BABILong push $I^*(Q)$ up and surface the Fano floor of Eq. (2). A second family of suites trades control for ecological validity. LongBench covers 21 multitask datasets [3], InfiniteBench drives average length past 100K with dependency-heavy tasks [139], and HELMET layers in model-based judging and few-shot prompting to repair the noisy n -gram signals and base-model incompatibility that plagued the earlier suites [133]. The third family, interactive memory benchmarks, aims at the agent layer. LOCOMO scores QA, summarization, and temporal reasoning over multi-session dialogues [74]. LongMemEval separates five memory abilities (extraction, cross-session and temporal reasoning, knowledge updates, and abstention) over $\sim 115K$ -token histories, where it sees $\sim 30\%$ drops on sustained interaction [118].

Two efforts come closest to compaction itself. SCBench studies long-context methods through a KV-cache-centric lens, spanning generation, compression, retrieval, and loading in a shared-context, multi-request setting [61]; KVPress [84] supplies a harness that standardizes KV compression so eviction and quantization methods all run behind one interface. We treat both as direct precursors and build on them rather than around them. NoLiMa adds a cautionary result: literal-match probes like NIAH systematically flatter the model. Strip the lexical overlap between needle and query so that recall has to pass through latent association, and accuracy falls far below the NIAH heatmap [78]. Read through Section 2, NIAH measures $I^*(Q)$ only in the easy regime where the answer-bearing bits are lexically marked, and NoLiMa shows that the bits compaction actually discards are the unmarked ones.

13.2 The missing benchmark

For all this coverage, no suite measures compaction the way the formalism requires, and it falls short in two related ways. The first is the budget axis. KV work reports tokens or GPU bytes, prompt work reports a compression ratio, recurrent work reports state

dimensions, and agent work reports store size. The units never line up, so the frontiers are incommensurable and each method’s budget is set by whoever wrote the paper. The second gap is more fundamental: none of these suites measures the capabilities that, by the formalism, tell good compaction from bad at equal B , namely whether a scheme knows what it dropped, whether it can get it back, and whether its confidence reflects what it kept. Post-compaction end-task accuracy is an integral over $\mathcal{D}(\theta)$. It averages away the lost bits instead of locating them, so it cannot say which unseen query will fail, and P-rev and P-q, the properties prediction (4) ties to repeated compaction, never get scored at all.

Our response is **COMPACT-Bench**, an extension of SCBench [61] and KVPress rather than a replacement: it keeps their task corpora and KV harness and supplies the axes and task families they leave out. Three pieces make it up.

The first is a shared budget axis. Every method lands on one bytes-per-token-of-history (BPT) scale, total retained memory in bytes over raw history tokens, the layer-agnostic form of $\text{rate}(Z)$, and is reported as a full accuracy-versus-budget frontier instead of one operating point. This makes the Eq. (2) floor visible on the page and the $1/I^*(Q)$ task-dependence of Section 2 measurable across layers.

The second piece is three task families that probe the capabilities end-task accuracy hides. **Loss attribution** checks whether a method knows what it dropped: after compaction, probe recall on a held-out set of history facts and compare the method’s own predicted retention against the truth, a direct test of its distortion surrogate. **Reversibility** checks whether dropped content can be re-derived on demand: issue a late query whose evidence was evicted and measure recovery, which separates retrieval-backed and archival schemes (P-rev holds) from eviction and summarization (where it does not). **Calibrated compaction confidence** checks whether a method’s stated confidence tracks its real post-compaction accuracy, so that an agent can abstain or re-expand instead of answering from a lossy state.

The third piece reports accuracy, latency, memory, and dollar cost together on one sheet, because a frontier that ignores decode latency or store cost conceals the very trade a deployment has to make. Since NoLiMa shows literal-match probes inflate quality [78], all retrieval-style items use latent-association needles rather than lexical ones, so the frontier reflects the unmarked bits that compaction destroys. The protocol’s sharpest test is prediction (4): plot end-task error against the number of compaction events, and it should climb super-linearly for irreversible summarization while staying flat for reversible memory, a contrast no existing suite can draw. Section 14 runs a first version of this protocol on the KV and agent layers.

14 A Reference Experiment

It is one thing to argue that the layers share a budget axis and a failure mode, another to watch it happen. So we ran two small studies on a single commodity GPU, an NVIDIA RTX 4060 with 8 GB, using Qwen2.5-1.5B-Instruct. We are not chasing state of the art here; we want to demonstrate the methodology the survey advocates, namely a common budget axis that makes heterogeneous methods comparable (Section 13) and a measurement of the

Benchmark	Type	Multi-budget?	Cross-layer?	Repeated compact.?
NIAH [51]	synthetic	○	○	○
RULER [46]	synthetic	●	○	○
BABILong [54]	synthetic	●	○	○
LongBench [3]	realistic	○	○	○
InfiniteBench [139]	realistic	○	○	○
HELMET [133]	realistic	●	○	○
LOCOMO [74]	interactive	○	○	●
LongMemEval [118]	interactive	○	○	●
SCBench [61]	KV	●	●	●
KVPress [84]	KV	●	○	○
COMPACT-Bench	all four	●	●	●

Table 5: Benchmarks for long-context and compaction evaluation against the properties the lens of Section 2 requires. *Multi-budget?* sweeps a memory-rate axis; *Cross-layer?* compares KV, prompt, architectural, and agent methods on one axis; *Repeated-compaction?* measures degradation under repeated compaction events. ● holds; ● partial; ○ absent.

repeated-compaction regime the field leaves untested. The absolute numbers are tied to this small model, but the result is the shape of the two curves, and both shapes carry over to larger scale. Code and configurations accompany the survey.

Every method sits on bytes-per-token-of-history (BPT), the bytes of retained memory state over the tokens of original context. The retained state is a KV cache in the end, so all four mechanisms collapse onto one currency: token eviction that keeps a fraction f costs f of the full per-token KV bytes, b -bit quantization costs $b/16$ of it, prompt or summary compression to m of n tokens costs m/n , and the full cache anchors the top. This is the axis COMPACT-Bench standardizes, and it is what puts a KV evictor and a quantizer on the same chart.

14.1 Experiment 1: the unified accuracy–budget frontier

The task is needle-in-a-haystack retrieval with natural filler text drawn from Wikitext: we plant a unique key–value needle at controlled depths (10% to 90%) in contexts of 2k to 8k tokens and ask for the value. We sweep six KV-compaction methods from Section 4, namely SnapKV [60], StreamingLLM [123], TOVA [85], the Knorm and expected-attention scorers, and a random-eviction control, each at five budgets. Accuracy is averaged over depths, lengths, and three trials, with the full cache anchoring the top at 100% budget.

Figure 5 reports the result over 1,395 generations. The full cache answers perfectly (accuracy 1.00). As the budget falls, the methods trace a frontier that slides toward zero, and below roughly a quarter of the full budget every method sits at or near zero accuracy. Eq. (2) predicts exactly this: the needle task concentrates its task-conditioned information content $I^*(Q)$ in a few tokens, so once the budget drops under that content no scorer can recover the answer, an instance of prediction (1). The random-eviction control does the

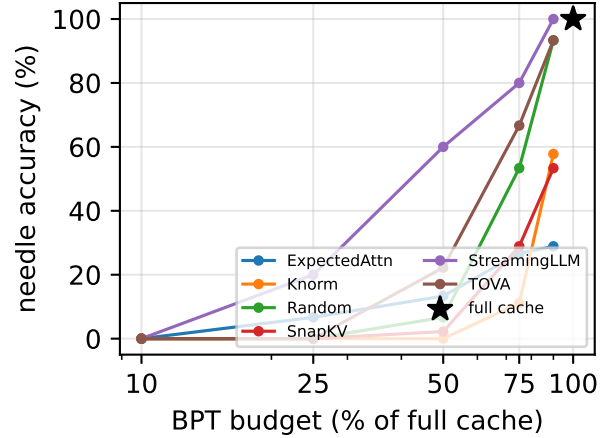


Figure 5: The accuracy–budget frontier on natural-filler needle retrieval with Qwen2.5–1.5B, six KV-compaction methods on one bytes-per-token-of-history axis. Accuracy collapses once the budget falls below the task’s information content, and the random-eviction control isolates the value each scorer adds.

diagnostic work. At high budget it stays competitive, because keeping most tokens keeps the needle by luck, but it collapses fastest as the budget tightens, so the gap between a real method and the random line measures how much its scoring buys at a given budget. The gaps are modest at this scale, and the ordering of methods is beside the point; what matters is the comparability the BPT axis delivers.

14.2 Experiment 2: error accumulation under repeated compaction

No single-turn benchmark can run this test, which is precisely why it matters. An agent reads a long document in chunks, accumulating twelve key–value facts spread throughout, and periodically compacts its working memory. We compare two operators as the number of compaction events grows. The irreversible operator overwrites working memory with an LLM-generated summary. The reversible operator keeps all chunks in an archive and retrieves the relevant chunk at query time, the Quest and MemGPT-archival discipline of P1. At the end we ask for all twelve facts and report recall.

Figure 6 shows the separation. The reversible operator holds recall near 0.95 at every compaction frequency, since retrieval can re-derive any dropped fact. The irreversible operator runs far below it, between 0.33 and 0.56, and is weakest at the highest compaction frequency, because each summary throws away facts the next summary can no longer see and the loss compounds. Two operators at the same average budget therefore part by roughly 0.5 in recall once memory is reused over a horizon, even though a single-turn needle test cannot tell them apart. This is what it means in practice to say that at equal budget, reversible compaction dominates.

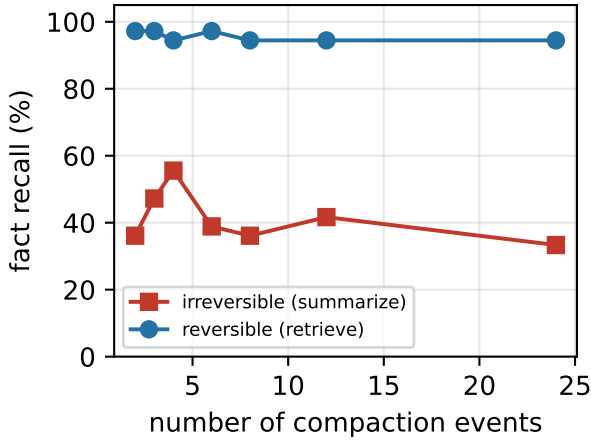


Figure 6: Fact recall against the number of compaction events for an irreversible (summarize) and a reversible (retrieve) operator. Reversible memory stays near 0.95; irreversible memory loses roughly half its facts at every frequency. The two look identical under single-turn evaluation and diverge only when compaction repeats.

Scale is not what makes either result land. The first shows that a single budget axis renders heterogeneous compaction methods comparable, and that accuracy collapses once the budget drops below the information content the task requires. The second shows that the regime agents actually live in, repeated compaction over a long horizon, splits reversible from irreversible operators that look identical under single-turn evaluation. Each is a falsifiable instance of the formalism, and each invites replication at larger scale.

15 Open Problems and Research Agenda

The formalism of Section 2 does more than unify the literature: it tells us which problems are worth solving and why. We close by setting out a prioritized agenda, ordered by impact times tractability. Each item is stated against the bound (2), the three properties (P-rev), (P-q), (P-fid), or the four predictions, so that “progress” has a precise meaning rather than another point on a borrowed long-context curve.

First, query-conditioned, reversible, multi-fidelity compaction at every layer. The single highest-leverage target follows directly from the bound: a query-agnostic operator pays a price of $H(Q)$ bits it cannot recover (consequence (iii)), and an irreversible one cannot re-spend its budget once the query arrives. The agenda is to build operators that satisfy (P-q) and (P-rev) simultaneously, with a (P-fid) profile that keeps a cheap lossy tier over a small recoverable high-fidelity tier, and to do so at each tier of the hierarchy, porting the query-conditioned, no-evict design of Quest [108] and the retrieval-backing of InfiniGen [57] up into prompt and agent memory. This is tractable because the components exist in isolation; the work is composing them and showing the predicted right-shift (Prediction 2) and flat error curve (Prediction 4).

Second, information-loss attribution and calibrated compaction confidence. Every layer today measures only end-task accuracy after the fact and cannot say what was dropped or predict failure on an unseen query. Because the surrogate scorers of Section 2 are all estimators of one distortion, they can in principle emit a calibrated estimate of residual $I^*(Q)$, a per-decision “compaction confidence” with provenance of loss. A method that knows it has fallen below $B < I^*(Q)$ can defer, fetch, or escalate fidelity; this attribution is the prerequisite for everything adaptive below.

Third, a composition map for stacking operators. Quantization, low-rank projection, eviction, and merging are treated as independent recipes, yet stacking them compounds or cancels distortion, and serving-level lossless reuse (RadixAttention; 145) directly conflicts with lossy budgeting (InfiniGen; 57). We need an honest empirical “operator algebra”, which pairs are synergistic, redundant, or destructive, within a layer (quant $\times$ low-rank $\times$ evict) and across layers (KV $\times$ prompt $\times$ agent), together with a treatment of the lossless-reuse versus lossy-budget tension as a first-class trade. We stress that this is a measured map, not a set of proven laws: distortion surrogates are not additive and composition order matters.

Fourth, a predictive compression scaling law. The bound makes the compression ratio at fixed quality scale as $1/I^*(Q)$, but $I^*(Q)$ is never measured directly. The open problem is a predictive law that estimates $I^*(Q)$, and hence the achievable ratio, as a function of model size, context length, head and layer structure, and task type, closing the gap between worst-case lower bounds ($\Theta(nd)$, sink necessity) and the reported 80–93% KV reductions. Such a law would turn budget selection from tuning into calculation.

Fifth, an episodic-to-semantic consolidation bridge and a stop rule. Within-task curation (ACON; 53, Context-Folding; 105) and cross-session memory (Mem0; 21, MemGPT; 86) use disjoint mechanisms with no principled promotion from a lossless, recoverable episodic tier to a lossy, abstracted semantic tier. The formalism supplies the missing pieces: promote a span only when its marginal I^* over the query distribution justifies abstracting it, and, because repeated irreversible summarization grows error super-linearly (Prediction 4), adopt an output-error stop rule for agent summarization in the spirit of Ada-KV’s bound.

Sixth, cheap, asynchronous, RL-stable agent self-curation. Reflection-, note-, and update-based memory is LLM-call-heavy and self-reinforcing: it spends calls that erode the savings and stores errors that bias all future retrieval. The target is curation that is asynchronous (off the critical path, as sleep-time compute [65] begins to be), learned rather than hand-triggered (MemAct; 141, MEM1; 148), and stable under RL so the reward does not collapse into degenerate forgetting.

Seventh, multimodal $\times$ multi-agent shared-memory compaction. Visual tokens are highly redundant and compress well (VisionZip; 127, PruMerge; 99); multi-agent memory carries provenance and access-control constraints (G-Memory; 136, MIRIX; 113). Nobody has compacted multimodal context shared across collaborating agents, where the governing quantity is cross-modal versus inter-message redundancy under per-agent query distributions, a clean multi-source instance of objective (1).

Eighth, adaptivity with guarantees. Budget, bit-width, rank, and ratio are allocated per layer, head, position, and turn by hand-tuned heuristics (PyramidKV; 11). The bound says the right allocation equalizes the residual $I^*(Q)$ across units, budgeting by marginal task-utility rather than uniform ratio, so the open problem is content- and query-adaptive allocation that comes with a guarantee on the resulting distortion rather than an empirical sweep.

Ninth, safety, auditability, and rollback of evolving memory. Self-editing memory that satisfies (P-rev) for utility also creates an attack surface: poisoning, leakage, and irreversible drift (AgentPoison; 18, MINJA; 30). Reversibility cuts both ways, enabling rollback and audit but also persistent injection, so the agenda is auditable, version-able compacted memory with a forgetting and revocation primitive, an area only beginning to be governed (surveyed in 66).

Two threads run through this list. At equal budget, an operator that can re-derive what it dropped (P-rev) and conditions on the query (P-q) beats one that discards for good, and the bound says why: only the reversible, conditioned operator escapes the $H(Q)$ penalty and the compounding error of repeated lossy summarization. And the field still has no cross-layer benchmark; methods report on budgets they choose for themselves over borrowed long-context suites, so their accuracy-versus-budget curves cannot be compared across granularities. Until a common budget axis exists, the agenda above will be measured in mismatched units, and the unification this survey proposes stays a framing rather than a tested science.

16 Conclusion

We have argued that memory compaction in language models and their agents is one problem wearing four disguises. Evicting a KV token, quantizing it, pruning a prompt span, bounding a recurrent state, and summarizing an agent’s trajectory are all instances of a single rate–distortion decision: what context-derived information to retain versus discard, at what fidelity, under a budget, so as to preserve task utility (Section 2). Reading the whole stack through this lens pays off three ways. It supplies a common currency: a budget axis on which a KV evictor and an agent summarizer can finally be compared. It exposes a single failure mode, the query-agnostic and irreversible loss of later-needed information, recurring at every layer and predicted by the same bound. And it licenses transfer: a forgetting curve becomes a KV prior, a no-eviction retrieval discipline becomes an agent-memory design, and an output-error bound becomes a stop rule for summarization (Section 10).

Two results stand out. At equal budget, a reversible operator beats an irreversible one: of two methods that keep the same number of bytes, the one that can recover what it dropped wins on the queries that depend on it, and the reference experiment shows that gap widening as the memory is reused (Section 14). And the field has no benchmark that holds one budget axis across all four layers while measuring repeated compaction; today’s suites report ratios at fixed quality on single-turn long context and leave the agentic regime, where compaction compounds, untested. We give such a benchmark in Section 13 and run a first version of it.

The agenda that follows (Section 15) is, in one sentence, to make compaction query-conditioned, reversible, and attributable: to keep what a future query will need, to recover what was dropped when

it is needed anyway, and to know what was lost. Progress will come not from a better heuristic at any one layer but from recognizing that the layers share an objective and measuring them the same way.

Limitations

This is a survey: it proposes a formalism and a benchmark protocol but does not provide large-scale empirical validation of either. The lower bound (Eq. 2) assumes a query-agnostic operator for its cleanest form and treats the task-conditioned information content $I^*(Q)$ as given; estimating $I^*(Q)$ for real tasks is itself open (Section 12). The cross-layer “operator algebra” we call for is, at present, an empirical composition map rather than a set of proven composition laws. Our reference experiment (Section 14) runs at reference scale (a small open model on a single commodity GPU), so its absolute numbers should be read as illustrations of the methodology, not as a leaderboard; the trends it isolates (the accuracy–budget frontier and error accumulation under repeated compaction) are the contribution, and they invite replication at scale. Finally, the field moves quickly and many cited works are recent preprints whose results await independent reproduction; we have flagged these where they bear on our claims, and reported compression ratios are drawn from heterogeneous setups that the normalization protocol of Section 13 is meant to reconcile rather than take at face value.

References

- [1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference. arXiv:2403.09054 [cs.LG] <https://arxiv.org/abs/2403.09054>
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. 2022. Flamingo: a Visual Language Model for Few-Shot Learning. arXiv:2204.14198 [cs.CV] <https://arxiv.org/abs/2204.14198>
- [3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508 [cs.CL] <https://arxiv.org/abs/2308.14508>
- [4] Ivana Balažević, Yuge Shi, Pinelopi Papalampidi, Rahma Chaabouni, Skanda Koppula, and Olivier J. Hénaff. 2024. Memory Consolidation Enables Long-Context Video Understanding. arXiv:2402.05861 [cs.CV] <https://arxiv.org/abs/2402.05861>
- [5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. 2024. xLSTM: Extended Long Short-Term Memory. arXiv:2405.04517 [cs.LG] <https://arxiv.org/abs/2405.04517>
- [6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. 2024. Titans: Learning to Memorize at Test Time. arXiv:2501.00663 [cs.LG] <https://arxiv.org/abs/2501.00663>
- [7] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2023. Token Merging: Your ViT But Faster. arXiv:2210.09461 [cs.CV] <https://arxiv.org/abs/2210.09461>
- [8] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. 2022. Improving language models by retrieving from trillions of tokens. arXiv:2112.04426 [cs.CL] <https://arxiv.org/abs/2112.04426>
- [9] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan Kelly. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. arXiv:2405.12981 [cs.LG] <https://arxiv.org/abs/2405.12981>

- [10] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. 2022. Recurrent Memory Transformer. arXiv:2207.06881 [cs.CL] <https://arxiv.org/abs/2207.06881>
- [11] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2025. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv:2406.02069 [cs.CL] <https://arxiv.org/abs/2406.02069>
- [12] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. 2024. Palu: Compressing KV-Cache with Low-Rank Projection. arXiv:2407.21118 [cs.AI] <https://arxiv.org/abs/2407.21118>
- [13] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Hung-Yueh Chiang, Yash Akhauri, Xilai Dai, Huiqiang Jiang, Yucheng Li, Luis Ceze, Kai-Chiang Wu, and Mohamed S. Abdelfattah. 2026. xKV: Cross-Layer KV-Cache Compression via Aligned Singular Vector Extraction. arXiv:2503.18893 [cs.CL] <https://arxiv.org/abs/2503.18893>
- [14] Vivek Chari, Guanghui Qin, and Benjamin Van Durme. 2025. KV-Distill: Nearly Lossless Learnable Context Compression for LLMs. arXiv:2503.10337 [cs.CL] <https://arxiv.org/abs/2503.10337>
- [15] Guoxin Chen, Zile Qiao, Xuanzhong Chen, Donglei Yu, Haotian Xu, Wayne Xin Zhao, Ruihua Song, Wenbiao Yin, Hui Feng Yin, Liwen Zhang, Kuan Li, Minpeng Liao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. 2026. IterResearch: Rethinking Long-Horizon Agents with Interaction Scaling. arXiv:2511.07327 [cs.AI] <https://arxiv.org/abs/2511.07327>
- [16] Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. 2025. SepLLM: Accelerate Large Language Models by Compressing One Segment into One Separator. arXiv:2412.12094 [cs.CL] <https://arxiv.org/abs/2412.12094>
- [17] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. arXiv:2403.06764 [cs.CV] <https://arxiv.org/abs/2403.06764>
- [18] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agent-Poison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. arXiv:2407.12784 [cs.LG] <https://arxiv.org/abs/2407.12784>
- [19] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. arXiv:2405.13792 [cs.CL] <https://arxiv.org/abs/2405.13792>
- [20] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. arXiv:2305.14788 [cs.CL] <https://arxiv.org/abs/2305.14788>
- [21] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413 [cs.CL] <https://arxiv.org/abs/2504.19413>
- [22] Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: efficient and robust context pruning for retrieval-augmented generation. arXiv:2501.16214 [cs.CL] <https://arxiv.org/abs/2501.16214>
- [23] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. arXiv:1901.02860 [cs.LG] <https://arxiv.org/abs/1901.02860>
- [24] Payal Das, Subhajit Chaudhury, Elliot Nelson, Igor Melnyk, Sarath Swaminathan, Sihui Dai, Aurélie Lozano, Georgios Kollias, Vijil Chenthamarakshan, Jiří, Navrátil, Soham Dan, and Pin-Yu Chen. 2024. Larimar: Large Language Models with Episodic Memory Control. arXiv:2403.11901 [cs.LG] <https://arxiv.org/abs/2403.11901>
- [25] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. 2024. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. arXiv:2402.19427 [cs.LG] <https://arxiv.org/abs/2402.19427>
- [26] Enrique Queipo de Llano, Álvaro Arroyo, Federico Barbero, Xiaowen Dong, Michael Bronstein, Yann LeCun, and Ravid Shwartz-Ziv. 2026. Attention Sinks and Compression Valleys in LLMs are Two Sides of the Same Coin. arXiv:2510.06477 [cs.LG] <https://arxiv.org/abs/2510.06477>
- [27] DeepSeek-AI. 2025. DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention. Technical report. DeepSeek Sparse Attention (DSA).
- [28] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Cheng-gang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiusi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuan Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xin-nan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Ma, Yongqiang Guo, Yuchen Zhu, Yuduan Wang, Yuheng Zou, Yukun Shi, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhinu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434 [cs.CL] <https://arxiv.org/abs/2405.04434>
- [29] Chenlong Deng, Zhisong Zhang, Kelong Mao, Shuaiyi Li, Xinting Huang, Dong Yu, and Zhicheng Dou. 2024. A Silver Bullet or a Compromise for Full Attention? A Comprehensive Study of Gist Token-based Context Compression. arXiv:2412.17483 [cs.CL] <https://arxiv.org/abs/2412.17483>
- [30] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. arXiv:2503.03704 [cs.LG] <https://arxiv.org/abs/2503.03704>
- [31] Pengfei Du. 2026. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. arXiv:2603.07670 [cs.AI] <https://arxiv.org/abs/2603.07670>
- [32] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanaky, Robert Osazuwa Ness, and Jonathan Larson. 2025. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 [cs.CL] <https://arxiv.org/abs/2404.16130>
- [33] Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Re. 2025. Cartridges: Lightweight and general-purpose long context representations via self-study. arXiv:2506.06266 [cs.CL] <https://arxiv.org/abs/2506.06266>
- [34] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. arXiv:2407.11550 [cs.CL] <https://arxiv.org/abs/2407.11550>
- [35] Zafeiros Fountas, Martin A. Benfeghoul, Adnan Omerjee, Fania Christopoulou, Gerasimos Lampouras, Haitham Bou-Ammar, and Jun Wang. 2025. Human-inspired Episodic Memory for Infinite Context LLMs. arXiv:2407.09450 [cs.AI] <https://arxiv.org/abs/2407.09450>
- [36] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cached Attention. arXiv:2403.19708 [cs.CL] <https://arxiv.org/abs/2403.19708>
- [37] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. 2025. SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs. arXiv:2410.13276 [cs.CL] <https://arxiv.org/abs/2410.13276>
- [38] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. arXiv:2310.01801 [cs.CL] <https://arxiv.org/abs/2310.01801>
- [39] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. arXiv:2307.06945 [cs.CL] <https://arxiv.org/abs/2307.06945>
- [40] Albert Gu and Tri Dao. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752 [cs.LG] <https://arxiv.org/abs/2312.00752>
- [41] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. 2025. When Attention Sink Emerges in Language Models: An Empirical View. arXiv:2410.10781 [cs.CL] <https://arxiv.org/abs/2410.10781>
- [42] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831 [cs.CL] <https://arxiv.org/abs/2405.14831>
- [43] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. arXiv:2502.14802 [cs.CL] <https://arxiv.org/abs/2502.14802>
- [44] Themistoklis Haris and Krzysztof Onak. 2025. Compression Barriers for Autoregressive Transformers. arXiv:2502.15955 [cs.DS] <https://arxiv.org/abs/2502.15955>

- [45] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2025. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv:2401.18079 [cs.LG] <https://arxiv.org/abs/2401.18079>
- [46] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654 [cs.CL] <https://arxiv.org/abs/2404.06654>
- [47] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. 2026. Memory in the Age of AI Agents. arXiv:2512.13564 [cs.CL] <https://arxiv.org/abs/2512.13564>
- [48] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. arXiv:2407.02490 [cs.CL] <https://arxiv.org/abs/2407.02490>
- [49] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. arXiv:2310.05736 [cs.CL] <https://arxiv.org/abs/2310.05736>
- [50] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. arXiv:2310.06839 [cs.CL] <https://arxiv.org/abs/2310.06839>
- [51] Greg Kamradt. 2023. Needle In A Haystack – Pressure Testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack. Software.
- [52] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. 2024. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527 [cs.LG] <https://arxiv.org/abs/2403.05527>
- [53] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2026. ACON: Optimizing Context Compression for Long-horizon LLM Agents. arXiv:2510.00615 [cs.AI] <https://arxiv.org/abs/2510.00615>
- [54] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. BABLong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack. arXiv:2406.10149 [cs.CL] <https://arxiv.org/abs/2406.10149>
- [55] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. In Search of Needles in a 11M Haystack: Recurrent Memory Finds What LLMs Miss. arXiv:2402.10790 [cs.CL] <https://arxiv.org/abs/2402.10790>
- [56] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] <https://arxiv.org/abs/2309.06180>
- [57] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. arXiv:2406.19707 [cs.LG] <https://arxiv.org/abs/2406.19707>
- [58] Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. 2025. A Survey on Large Language Model Acceleration based on KV Cache Management. arXiv:2412.19442 [cs.AI] <https://arxiv.org/abs/2412.19442>
- [59] Yucheng Li. 2023. Unlocking Context Constraints of LLMs: Enhancing Context Efficiency of LLMs with Self-Information-Based Content Filtering. arXiv:2304.12102 [cs.CL] <https://arxiv.org/abs/2304.12102>
- [60] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469 [cs.CL] <https://arxiv.org/abs/2404.14469>
- [61] Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. 2025. SCBench: A KV Cache-Centric Analysis of Long-Context Methods. arXiv:2412.10319 [cs.CL] <https://arxiv.org/abs/2412.10319>
- [62] Zongqian Li, Yinlong Liu, Yixuan Su, and Nigel Collier. 2024. Prompt Compression for Large Language Models: A Survey. arXiv:2410.12388 [cs.CL] <https://arxiv.org/abs/2410.12388>
- [63] Zongqian Li, Yixuan Su, and Nigel Collier. 2024. 500xCompressor: Generalized Prompt Compression for Large Language Models. arXiv:2408.03094 [cs.CL] <https://arxiv.org/abs/2408.03094>
- [64] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meir, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887 [cs.CL] <https://arxiv.org/abs/2403.19887>
- [65] Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. 2025. Sleep-time Compute: Beyond Inference Scaling at Test-time. arXiv:2504.13171 [cs.AI] <https://arxiv.org/abs/2504.13171>
- [66] Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. 2026. A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle. arXiv:2604.16548 [cs.CR] <https://arxiv.org/abs/2604.16548>
- [67] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. arXiv:2405.14366 [cs.CL] <https://arxiv.org/abs/2405.14366>
- [68] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172 [cs.CL] <https://arxiv.org/abs/2307.03172>
- [69] Yanyu Liu, Jingying Fu, Sixiang Liu, Yitian Zou, You Fu, Jiehan Zhou, and Shouhua Zhang. 2025. KV Cache Compression for Inference Efficiency in LLMs: A Review. arXiv:2508.06297 [cs.DC] <https://arxiv.org/abs/2508.06297>
- [70] Yuhuan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. arXiv:2310.07240 [cs.NI] <https://arxiv.org/abs/2310.07240>
- [71] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. arXiv:2305.17118 [cs.LG] <https://arxiv.org/abs/2305.17118>
- [72] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750 [cs.CL] <https://arxiv.org/abs/2402.02750>
- [73] Enze Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Neo Y. Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. 2025. MoBA: Mixture of Block Attention for Long-Context LLMs. arXiv:2502.13189 [cs.LG] <https://arxiv.org/abs/2502.13189>
- [74] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753 [cs.CL] <https://arxiv.org/abs/2402.17753>
- [75] Fanxu Meng, Pingzhi Tang, Xiaojuan Tang, Zengwei Yao, Xing Sun, and Muhuan Zhang. 2025. TransMLA: Multi-Head Latent Attention Is All You Need. arXiv:2502.07864 [cs.LG] <https://arxiv.org/abs/2502.07864>
- [76] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and Editing Factual Associations in GPT. arXiv:2202.05262 [cs.CL] <https://arxiv.org/abs/2202.05262>
- [77] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2023. Mass-Editing Memory in a Transformer. arXiv:2210.07229 [cs.CL] <https://arxiv.org/abs/2210.07229>
- [78] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. NoLiMa: Long-Context Evaluation Beyond Literal Matching. arXiv:2502.05167 [cs.CL] <https://arxiv.org/abs/2502.05167>
- [79] Amirkeivan Mohtashami and Martin Jaggi. 2023. Landmark Attention: Random-Access Infinite Context Length for Transformers. arXiv:2305.16300 [cs.CL] <https://arxiv.org/abs/2305.16300>
- [80] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2024. Learning to Compress Prompts with Gist Tokens. arXiv:2304.08467 [cs.CL] <https://arxiv.org/abs/2304.08467>
- [81] Tsendsuren Munkhdalai, Manaal Faruqi, and Siddharth Gopal. 2024. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv:2404.07143 [cs.CL] <https://arxiv.org/abs/2404.07143>
- [82] Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M. Ponti. 2024. Dynamic Memory Compression: Retrofitting LLMs for Accelerated Inference. arXiv:2403.09636 [cs.CL] <https://arxiv.org/abs/2403.09636>
- [83] Eshaan Nichani, Jason D. Lee, and Alberto Bietti. 2024. Understanding Factual Recall in Transformers via Associative Memories. arXiv:2412.06538 [cs.LG] <https://arxiv.org/abs/2412.06538>
- [84] NVIDIA. 2024. KVPress: LLM KV Cache Compression Made Easy. <https://github.com/NVIDIA/kvpress>. Software library.
- [85] Matalen Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. 2024. Transformers are Multi-State RNNs. arXiv:2401.06104 [cs.CL] <https://arxiv.org/abs/2401.06104>

- [86] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] <https://arxiv.org/abs/2310.08560>
- [87] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. arXiv:2403.12968 [cs.CL] <https://arxiv.org/abs/2403.12968>
- [88] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442 [cs.HC] <https://arxiv.org/abs/2304.03442>
- [89] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. arXiv:2311.18677 [cs.AR] <https://arxiv.org/abs/2311.18677>
- [90] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Krantthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. RWKV: Reinventing RNNs for the Transformer Era. arXiv:2305.13048 [cs.CL] <https://arxiv.org/abs/2305.13048>
- [91] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. arXiv:2405.04437 [cs.LG] <https://arxiv.org/abs/2405.04437>
- [92] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079 [cs.DC] <https://arxiv.org/abs/2407.00079>
- [93] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lili-crap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv:1911.05507 [cs.LG] <https://arxiv.org/abs/1911.05507>
- [94] Yuval Ran-Milo. 2026. Attention Sinks Are Provably Necessary in Softmax Transformers: Evidence from Trigger-Conditional Tasks. arXiv:2603.11487 [cs.LG] <https://arxiv.org/abs/2603.11487>
- [95] Preston Rasmussen, Pavlo Palychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 [cs.CL] <https://arxiv.org/abs/2501.13956>
- [96] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. 2025. Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling. arXiv:2406.07522 [cs.CL] <https://arxiv.org/abs/2406.07522>
- [97] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv:2401.18059 [cs.CL] <https://arxiv.org/abs/2401.18059>
- [98] Shivam Shandilya, Menglin Xia, Supriyo Ghosh, Huiqiang Jiang, Jue Zhang, Qianhui Wu, and Victor Rühle. 2024. TACO-RL: Task Aware Prompt Compression Optimization with Reinforcement Learning. arXiv:2409.13035 [cs.CL] <https://arxiv.org/abs/2409.13035>
- [99] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. 2026. LLaVA-PrMerge: Adaptive Token Reduction for Efficient Large Multimodal Models. arXiv:2403.15388 [cs.CV] <https://arxiv.org/abs/2403.15388>
- [100] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. arXiv:2303.06865 [cs.LG] <https://arxiv.org/abs/2303.06865>
- [101] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI] <https://arxiv.org/abs/2303.11366>
- [102] Charlie Snell, Dan Klein, and Ruiqi Zhong. 2022. Learning by Distilling Context. arXiv:2209.15189 [cs.CL] <https://arxiv.org/abs/2209.15189>
- [103] Enxin Song, Wenhao Chai, Guan hong Wang, Yucheng Zhang, Haoyang Zhou, Feiyang Wu, Haozhe Chi, Xun Guo, Tian Ye, Yanting Zhang, Yan Lu, Jenq-Neng Hwang, and Gaoang Wang. 2024. MovieChat: From Dense Token to Sparse Memory for Long Video Understanding. arXiv:2307.16449 [cs.CV] <https://arxiv.org/abs/2307.16449>
- [104] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. 2025. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference. arXiv:2410.21465 [cs.LG] <https://arxiv.org/abs/2410.21465>
- [105] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967 [cs.CL] <https://arxiv.org/abs/2510.11967>
- [106] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. arXiv:2405.05254 [cs.CL] <https://arxiv.org/abs/2405.05254>
- [107] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. 2024. RazorAttention: Efficient KV Cache Compression Through Retrieval Heads. arXiv:2407.15891 [cs.LG] <https://arxiv.org/abs/2407.15891>
- [108] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. arXiv:2406.10774 [cs.CL] <https://arxiv.org/abs/2406.10774>
- [109] Naftali Tishby, Fernando C. Pereira, and William Bialek. 2000. The information bottleneck method. arXiv:physics/0004057 [physics.data-an] <https://arxiv.org/abs/physics/0004057>
- [110] Dezhao Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. 2024. VL-Cache: Sparsity and Modality-Aware KV Cache Compression for Vision-Language Model Inference Acceleration. arXiv:2410.23317 [cs.CV] <https://arxiv.org/abs/2410.23317>
- [111] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient Large Language Models: A Survey. arXiv:2312.03863 [cs.CL] <https://arxiv.org/abs/2312.03863>
- [112] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. 2024. LOOK-M: Look-Once Optimization in KV Cache for Efficient Multimodal Long-Context Inference. arXiv:2406.18139 [cs.CL] <https://arxiv.org/abs/2406.18139>
- [113] Yu Wang and Xi Chen. 2025. MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv:2507.07957 [cs.CL] <https://arxiv.org/abs/2507.07957>
- [114] Yu Wang, Yifan Gao, Xiuxi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. 2024. MEMORYLLM: Towards Self-Updatable Large Language Models. arXiv:2402.04624 [cs.CL] <https://arxiv.org/abs/2402.04624>
- [115] Yihang Wang, Xu Huang, Bowen Tian, Yueyang Su, Lei Yu, Huaming Liao, Yixing Fan, Jiafeng Guo, and Xueqi Cheng. 2025. QUITO-X: A New Perspective on Context Compression from the Information Bottleneck Theory. arXiv:2408.10497 [cs.CL] <https://arxiv.org/abs/2408.10497>
- [116] Zhiruo Wang, Jun Araki, Zhengbao Jiang, Md Rizwan Parvez, and Graham Neubig. 2023. Learning to Filter Context for Retrieval-Augmented Generation. arXiv:2311.08377 [cs.CL] <https://arxiv.org/abs/2311.08377>
- [117] Kaiyue Wen, Xingyu Dang, and Kaifeng Lyu. 2024. RNNs are not Transformers (Yet): The Key Bottleneck on In-context Retrieval. arXiv:2402.18510 [cs.LG] <https://arxiv.org/abs/2402.18510>
- [118] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813 [cs.CL] <https://arxiv.org/abs/2410.10813>
- [119] Jialong Wu, Zhenglin Wang, Linhai Zhang, Yilong Lai, Yulan He, and Deyu Zhou. 2025. SCOPE: Optimizing Key-Value Cache Compression in Long-context Generation. arXiv:2412.13649 [cs.CL] <https://arxiv.org/abs/2412.13649>
- [120] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. 2026. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313 [cs.CL] <https://arxiv.org/abs/2509.13313>
- [121] Yuhuai Wu, Markus N. Rabe, DeLeshley Hutchins, and Christian Szegedy. 2022. Memorizing Transformers. arXiv:2203.08913 [cs.LG] <https://arxiv.org/abs/2203.08913>
- [122] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. arXiv:2410.10819 [cs.CL] <https://arxiv.org/abs/2410.10819>
- [123] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453 [cs.CL] <https://arxiv.org/abs/2309.17453>
- [124] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2023. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. arXiv:2310.04408 [cs.CL] <https://arxiv.org/abs/2310.04408>
- [125] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.CL] <https://arxiv.org/abs/2502.12110>
- [126] Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. arXiv:2405.12532 [cs.CL] <https://arxiv.org/abs/2405.12532>
- [127] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. 2026. VisionZip: Longer is Better but Not Necessary in Vision Language Models. arXiv:2412.04467 [cs.CV] <https://arxiv.org/abs/2412.04467>
- [128] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. arXiv:2412.06464 [cs.CL] <https://arxiv.org/abs/2412.06464>

- [129] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. 2024. Gated Linear Attention Transformers with Hardware-Efficient Training. arXiv:2312.06635 [cs.LG] <https://arxiv.org/abs/2312.06635>
- [130] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2025. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484 [cs.LG] <https://arxiv.org/abs/2406.06484>
- [131] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444 [cs.LG] <https://arxiv.org/abs/2405.16444>
- [132] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>
- [133] Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2025. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. arXiv:2410.02694 [cs.CL] <https://arxiv.org/abs/2410.02694>
- [134] Chanwoong Yoon, Taewho Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing Retrieved Documents Actively for Question Answering. arXiv:2407.09014 [cs.CL] <https://arxiv.org/abs/2407.09014>
- [135] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv:2502.11089 [cs.CL] <https://arxiv.org/abs/2502.11089>
- [136] Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. arXiv:2506.07398 [cs.MA] <https://arxiv.org/abs/2506.07398>
- [137] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. 2024. Long Context Compression with Activation Beacon. arXiv:2401.03462 [cs.CL] <https://arxiv.org/abs/2401.03462>
- [138] Tianyi Zhang, Jonah Yi, Zhaozhuo Xu, and Anshumali Shrivastava. 2024. KV Cache is 1 Bit Per Channel: Efficient Large Language Model Inference with Coupled Quantization. arXiv:2405.03917 [cs.LG] <https://arxiv.org/abs/2405.03917>
- [139] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens. arXiv:2402.13718 [cs.CL] <https://arxiv.org/abs/2402.13718>
- [140] Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. 2024. CaM: Cache Merging for Memory-efficient LLMs Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. <https://proceedings.mlr.press/v235/zhang24n.html>
- [141] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. 2026. Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks. arXiv:2510.12635 [cs.AI] <https://arxiv.org/abs/2510.12635>
- [142] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048 [cs.LG] <https://arxiv.org/abs/2306.14048>
- [143] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. arXiv:2308.10144 [cs.LG] <https://arxiv.org/abs/2308.10144>
- [144] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit Quantization for Efficient and Accurate LLM Serving. arXiv:2310.19102 [cs.LG] <https://arxiv.org/abs/2310.19102>
- [145] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>
- [146] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. MemoryBank: Enhancing Large Language Models with Long-Term Memory. arXiv:2305.10250 [cs.CL] <https://arxiv.org/abs/2305.10250>
- [147] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC] <https://arxiv.org/abs/2401.09670>
- [148] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. arXiv:2506.15841 [cs.CL] <https://arxiv.org/abs/2506.15841>

A Master Method-by-Axis Table

Table 6 classifies representative methods from across the survey along the seven axes of Section 3. Abbreviations. Granularity: tok (token/KV entry), pg (page/block), bit (bit-width), rk (rank), hd (head/layer), span (NL span), gist (soft vector), st (recurrent state), fact (semantic item), vtok (visual token). Lifecycle: pre (pretraining), pf (prefill), dec (decode), srv (serving), task (within-task), x-task (cross-task). Rev.: ● reversible, ◐ partial, ○ irreversible. Adapt.: agn (query-agnostic), qry (query-conditioned), lrn (learned reward). Learn.: free (training-free), adpt (adapter/post-train), scr (from scratch), RL, ctrl (LLM controller).

Method	Layer	Granularity	Lifecycle	Rev.	Adapt.	Learn.	Mechanism
StreamingLLM [123]	KV	tok	dec	○	agn	free	evict (sink+window)
H2O [142]	KV	tok	dec	○	agn	free	evict (attn score)
SnapKV [60]	KV	tok	pf	○	qry	free	evict (obs. window)
PyramidKV [11]	KV	tok/hd	pf	○	agn	free	evict (layer budget)
Ada-KV [34]	KV	tok/hd	pf	○	agn	free	evict (error bound)
Quest [108]	KV	pg	dec	●	qry	free	select (keep all)
KIVI [72]	KV	bit	dec	◐	agn	free	quantize
Palu [12]	KV	rk	pf	◐	agn	adpt	low-rank
MLA [28]	KV	rk	pre	◐	agn	scr	low-rank (latent)
CaM [140]	KV	tok	dec	○	agn	free	merge
LLMLingua [49]	Prompt	span	pf	○	agn	free	prune (perplexity)
LongLLMLingua [50]	Prompt	span	pf	○	qry	free	prune (query-aware)
Gisting [80]	Prompt	gist	pf	○	agn	adpt	encode-to-latent
ICAE [39]	Prompt	gist	pf	○	agn	adpt	autoencode
xRAG [19]	Prompt	gist	pf	○	agn	adpt	embed-as-token
Cartridges [33]	Prompt	param	x-task	◐	agn	adpt	offline distill
RMT [10]	Arch	st	pre	○	agn	scr	segment recurrence
Infini-attn [81]	Arch	st	pre	○	agn	scr	compressive memory
Mamba [40]	Arch	st	pre	○	agn	scr	SSM constant state
Titans [6]	Arch	st	pre	◐	qry	scr	test-time memory
DMC [82]	Arch	tok	pre	○	ltn	adpt	learned accumulate
MemGPT [86]	Agent	fact	task	●	qry	ctrl	page / archive
ACON [53]	Agent	span	task	○	agn	ctrl	guideline summarize
MEM1 [148]	Agent	span	task	○	ltn	RL	learned consolidate
Mem0 [21]	Agent	fact	x-task	◐	qry	ctrl	add/update/delete
MemoryBank [146]	Agent	fact	x-task	◐	agn	ctrl	decay curve
RAPTOR [97]	Agent	fact	x-task	●	qry	ctrl	tree summary + retrieve
NSA [135]	Sparse	pg	pre	◐	qry	scr	learned sparse attn
MoBA [73]	Sparse	pg	pre	◐	qry	scr	block routing
DuoAttention [122]	Sparse	hd	dec	◐	agn	adpt	retrieval/stream heads
MInference [48]	Sparse	pg	pf	◐	qry	free	dynamic prefill sparsity
ToMe [7]	MM	vtok	pf	○	agn	free	merge visual tokens
LOOK-M [112]	MM	tok/vtok	pf	○	agn	free	evict + merge
PagedAttention [56]	Sys	tok	srv	●	agn	free	paged reuse
InfiniGen [57]	Sys	tok	srv	◐	qry	free	offload + select

Table 6: Representative methods classified along the seven axes of Section 3. The table makes the survey’s organizing claim concrete: methods that share no venue or vocabulary nonetheless occupy nearby points in this space, and the columns that vary most, namely granularity, lifecycle, reversibility, and query-adaptivity, are exactly those the rate–distortion lens predicts are decisive.